AI-Based Trip Planner -Web Application

A thesis submitted in partial fulfilment of the requirements

for the award of the degree of

Bachelor of Technology

By

Oinam Dinibash Singh	(2201CS0105)
Laba Moirangthem	(2201CS0104)
Deemson Pukhrambam	(2201CS0118)
Mathiuthailiu Panmei	(2301CS0302)

Under the Guidance of

Mr. Golmei Shaheamlung,

Assistant Professor, CSE,MTU

DEPARTMENT OF
COMPUTER SCIENCE & ENGINEERING

MANIPUR TECHNICAL UNIVERSITY

June, 2025

MANIPUR TECHNICAL UNIVERSITY

(A University Established under the Manipur Technical University Act, 2016)
Imphal, Manipur – 795004
Website: www. Mtu.ac.in

BONA FIDE CERTIFICATE

This is to certify that the thesis entitled "AI-Based Trip Planner Web Application" submitted by

Oinam Dinibash Singh (2201CS0105)

Laba Moirangthem (2201CS0104)

Deemson Pukhrambam (2201CS0118)

Mathiuthailiu Panmei (2301CS0302)

to the Department of Computer Science & Engineering, in partial fulfillment of the requirements for the award of the Bachelor of Technology degree in Computer Science and Engineering, is a genuine and original work carried out by them under my supervision during the academic year 2024–2025.

The project embodies the results of their own research and development work and has not been submitted, either in part or in full, for the award of any other degree or diploma of this university or any other institution. The work has been carried out with sincerity, academic integrity, and adherence to research ethics.

I consider this work worthy of consideration for the partial fulfillment of the requirements for the said degree.

Supervisor
Mr. Golmei Shaheamlung,
Assistant professor,
Department of Computer Science and Engineering
Manipur Technical University

Head of Department
Mrs. Aerena Tayenjam
HOD and Assistant professor,
Department of Computer Science and Engineering
Manipur Technical University

MANIPUR TECHNICAL UNIVERSITY

(A University Established under the Manipur Technical University Act, 2016)
Imphal, Manipur – 795004
Website: www. Mtu.ac.in

DECLARATION

I declare that this project entitled "AI-Based Trip Planner Web Application", submitted in partial fulfillment of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of original work carried out jointly by us under the supervision of Golmei Shaheamlung, Assistant Professor, Department of Computer Science and Engineering, and has not formed the basis for the award of any other degree or diploma in this or any other Institution or University.

In keeping with ethical practices in reporting scientific information, due acknowledgements have been made wherever the findings of others have been cited.

Oinam Dinibash Singh
(2201CS0105)

Department of Computer Science and Engineering
Manipur Technical University

Date:
Place: Imphal

MANIPUR TECHNICAL UNIVERSITY

(A University Established under the Manipur Technical University Act, 2016)
Imphal, Manipur – 795004
Website: www. Mtu.ac.in

DECLARATION

I declare that this project entitled "AI-Based Trip Planner Web Application", submitted in partial fulfillment of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of original work carried out jointly by us under the supervision of Golmei Shaheamlung, Assistant Professor, Department of Computer Science and Engineering, and has not formed the basis for the award of any other degree or diploma in this or any other Institution or University.

In keeping with ethical practices in reporting scientific information, due acknowledgements have been made wherever the findings of others have been cited.

Deemson Pukhrambam
(2201CS0118)
Department of Computer Science and Engineering
Manipur Technical University

Date:
Place: Imphal

MANIPUR TECHNICAL UNIVERSITY

(A University Established under the Manipur Technical University Act, 2016)
Imphal, Manipur – 795004
Website: www. Mtu.ac.in

DECLARATION

I declare that this project entitled "AI-Based Trip Planner Web Application", submitted in partial fulfillment of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of original work carried out jointly by us under the supervision of Golmei Shaheamlung, Assistant Professor, Department of Computer Science and Engineering, and has not formed the basis for the award of any other degree or diploma in this or any other Institution or University.

In keeping with ethical practices in reporting scientific information, due acknowledgements have been made wherever the findings of others have been cited.

Laba Moirangthem
(2201CS0104)
Department of Computer Science and Engineering
Manipur Technical University

Date:
Place: Imphal

MANIPUR TECHNICAL UNIVERSITY

(A University Established under the Manipur Technical University Act, 2016)
Imphal, Manipur – 795004
Website: www. Mtu.ac.in

DECLARATION

I declare that this project entitled "AI-Based Trip Planner Web Application", submitted in partial fulfillment of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of original work carried out jointly by us under the supervision of Golmei Shaheamlung, Assistant Professor, Department of Computer Science and Engineering, and has not formed the basis for the award of any other degree or diploma in this or any other Institution or University.

In keeping with ethical practices in reporting scientific information, due acknowledgements have been made wherever the findings of others have been cited.

Mathiuthailiu Panmei
(2301CS0302)
Department of Computer Science and Engineering
Manipur Technical University

Date:
Place: Imphal

MANIPUR TECHNICAL UNIVERSITY

(A University Established under the Manipur Technical University Act, 2016)
Imphal, Manipur – 795004
Website: www. Mtu.ac.in

ACKNOWLEDGEMENT

We sincerely express our gratitude to everyone who supported us in the successful completion of our project entitled “AI-Based Trip Planner Web Application.”

We are deeply thankful to our Project Supervisor, Golmei Shaheamlung, Assistant Professor, Department of Computer Science and Engineering, Manipur Technical University, for his valuable guidance, continuous support and encouragement throughout the project.

We also extend our sincere thanks to W. Chandbahu Singh, Vice Chancellor of Manipur Technical University, for providing a motivating academic environment and necessary facilities.

We are grateful to Aerena Tayenjam, Head of the Department, and Aerena Tayenjam, Project Coordinator, for their encouragement and support during the completion of this work.

We also thank all faculty members, friends and our family members for their constant motivation and support. Above all, we thank the Almighty for giving us the strength and determination to complete this project successfully.

Sincerely,
Oinam Dinibash Singh
Laba Moirangthem
Deemson Pukhrambam
Mathiuthailiu Panmei

Date:
Place: Imphal

ABSTRACT

The advancement of artificial intelligence and machine learning has significantly transformed the way people plan and manage travel experiences. Traditional trip planning often involves extensive manual research related to destinations, budgeting, accommodations, climate conditions, transportation, and itinerary scheduling, making the process time-consuming and inefficient. To overcome these challenges, this project presents the development of an AI-Based Trip Planner Web Application that provides intelligent and personalized travel recommendations using machine learning techniques and modern web technologies.

The application is developed using the Django framework for backend development and utilizes Scikit-learn to implement the recommendation engine. The system is designed to generate customized travel plans based on user inputs such as budget, number of travel days, preferred climate, travel type, and destination interests. By analyzing these parameters, the application recommends suitable destinations and generates optimized itineraries tailored to individual user preferences.

The platform incorporates a user-friendly and responsive interface that allows users to register, log in, save travel plans, manage itineraries, and revisit previous trips. A soft-delete feature is implemented to prevent accidental data loss while maintaining flexibility in trip management. The frontend is developed using modern web technologies including HTML, CSS, and JavaScript to ensure accessibility across multiple devices such as desktops, tablets, and smartphones.

A major component of the system is the machine learning recommendation module, which processes travel-related datasets to classify destinations according to categories such as adventure, nature, family, cultural, luxury, and budget travel. The model analyzes relationships between user preferences and destination attributes to provide relevant recommendations with improved accuracy and efficiency. Data preprocessing techniques including cleaning, encoding, and feature extraction are used to improve model performance and reliability.

The application also includes dynamic itinerary generation, where day-wise schedules are automatically created based on the selected destination and trip duration. The itinerary contains suggested tourist attractions, activities, estimated expenses, and travel guidance, reducing the manual effort required in conventional planning methods.

The project follows a modular and scalable system architecture using Django's Model-View-Template (MVT) design pattern, ensuring maintainability and future extensibility. Comprehensive testing methods including unit testing, integration testing, and usability testing are performed to ensure system reliability, security, and performance.

Overall, the AI-Based Trip Planner Web Application demonstrates how artificial intelligence can enhance the tourism industry by delivering smart, efficient, and personalized travel planning solutions. The system simplifies trip organization, improves user experience, and provides a foundation for future enhancements such as real-time booking integration, chatbot assistance, and advanced recommendation systems.

Table of Contents

List of Tables

List of Figures

Chapter 1: Introduction

1.1 Background

The travel and tourism industry has undergone a major transformation over the last decade due to the rapid growth of digital technologies, internet accessibility, and artificial intelligence. Modern travelers increasingly depend on online platforms to discover destinations, compare travel costs, analyze weather conditions, book accommodations, and create travel itineraries. The emergence of smart technologies has shifted the tourism sector from traditional travel agency models toward highly personalized digital travel ecosystems. Travelers today expect customized experiences that align with their financial capacity, available vacation time, interests, and lifestyle preferences.

Despite the availability of numerous online travel platforms, most existing systems still rely heavily on manual exploration and generic recommendation mechanisms. Users often spend hours researching destinations, comparing budgets, checking climate conditions, reading reviews, and organizing transportation schedules. This process becomes more difficult when travelers must balance multiple constraints such as limited budgets, short vacation periods, climate preferences, and preferred travel categories such as adventure, cultural tourism, family trips, or luxury vacations.

Traditional travel agencies generally offer pre-designed travel packages that may not fully satisfy individual user expectations. These packages are usually static and lack the flexibility required for personalized travel planning. Furthermore, many travel recommendation websites provide destination suggestions based only on popularity rather than user-specific requirements. As a result, travelers frequently experience information overload and decision fatigue while trying to select suitable destinations and organize realistic itineraries.

The advancement of artificial intelligence and machine learning technologies provides an opportunity to solve these challenges by automating and optimizing the travel planning process. Intelligent recommendation systems are capable of analyzing large datasets and identifying patterns between user preferences and destination attributes. Machine learning models can process multiple constraints simultaneously and generate accurate, personalized travel recommendations within seconds. These technologies allow travel systems to move beyond generic suggestions and provide adaptive, data-driven solutions tailored to each traveler.

This project introduces an AI-Based Trip Planner Web Application, developed using the Django framework and powered by machine learning algorithms implemented through Scikit-learn. The system aims to provide a complete travel planning ecosystem capable of generating intelligent destination recommendations and dynamic itineraries based on user-defined constraints such as budget, trip duration, climate preference, and travel type. The platform combines artificial intelligence with modern web technologies to simplify travel planning, improve user experience, and reduce the effort associated with conventional trip organization methods.

1.2 Problem Statement

Planning a trip manually is often a complicated and time-consuming process that involves collecting information from multiple sources. Travelers must search for destinations, estimate expenses, analyze weather conditions, compare accommodations, organize transportation, and create day-wise schedules. This process requires significant effort and can become overwhelming, especially for users with limited travel experience or strict constraints.

One of the major issues in existing travel planning systems is the lack of personalization. Many platforms provide generic recommendations based on trending destinations rather than the unique preferences of users. Such systems fail to consider important parameters including travel budget, number of travel days, preferred climate, travel interests, and trip purpose. Consequently, users may receive recommendations that are financially impractical, geographically inconvenient, or misaligned with their interests.

Another major challenge is information overload. The internet contains an enormous amount of travel-related content, including blogs, advertisements, reviews, and booking websites. While access to information is beneficial, excessive data often makes decision-making more difficult. Users frequently struggle to filter relevant information and identify destinations that truly match their requirements. Additionally, manually organizing travel itineraries requires extensive planning and coordination, increasing the possibility of scheduling conflicts and inaccurate budget estimation.

Existing travel systems also lack intelligent automation capabilities. Most applications require users to manually compare destinations and create schedules themselves. There is limited use of machine learning techniques for predictive recommendation and personalized itinerary generation. Furthermore, many systems do not provide secure trip management functionalities such as saved itineraries, account-based personalization, and recoverable data storage mechanisms.

Therefore, there is a significant need for an intelligent, automated, and user-centric travel planning system capable of synthesizing multiple user constraints into actionable and personalized travel plans. Such a system should reduce manual effort, improve recommendation accuracy, and provide a seamless planning experience while maintaining scalability, security, and usability.

1.3 Objectives

The primary aim of this project is to design and develop an intelligent web-based travel planning system that leverages machine learning and modern web technologies to provide personalized travel recommendations and itinerary management. The project focuses on enhancing user convenience, reducing planning complexity, and improving overall travel decision-making efficiency.

The major objectives of the project are as follows:

1. To Design and Develop a Web-Based Travel Planning Platform

The project aims to build a fully functional web application that acts as a centralized platform for travel planning activities. The application provides an interactive and user-

friendly environment where users can explore destinations, receive recommendations, generate itineraries, and manage saved trips. The platform is designed using responsive web technologies to ensure accessibility across desktops, tablets, and smartphones.

2. To Implement a Machine Learning-Based Recommendation System

A key objective of the project is to develop a machine learning recommendation engine capable of predicting suitable travel destinations based on user-defined constraints such as budget, number of travel days, preferred climate, and travel type. The recommendation system uses processed travel datasets and classification techniques to generate personalized and relevant destination suggestions.

3. To Develop a Dynamic Itinerary Generation Engine

The project seeks to automate itinerary planning by generating realistic day-by-day schedules for selected destinations. The itinerary engine provides users with suggested tourist attractions, activities, travel timings, estimated expenses, and travel guidance. This feature minimizes manual planning effort and enhances trip organization efficiency.

4. To Ensure Secure User Authentication and Data Management

The system incorporates secure authentication mechanisms to protect user accounts and travel data. Users can create accounts, log in securely, save generated itineraries, and manage travel history. A soft-delete mechanism is implemented to allow recovery of deleted trips and improve data safety.

5. To Create a Scalable and Maintainable Architecture

Another important objective is to develop the system using a modular and scalable architecture that supports future expansion. The application follows Django’s Model-View-Template (MVT) architecture to ensure maintainability, clean code organization, and efficient backend operations.

6. To Enhance User Experience Through Interactive UI/UX Design

The project aims to provide a visually appealing and intuitive user interface that improves usability and accessibility. The design focuses on easy navigation, responsive layouts, and interactive travel planning features to create a smooth user experience.

1.4 Scope of the Project

The scope of this project includes the complete design, development, and implementation of a prototype-level AI-powered travel planning web application focusing primarily on domestic tourism within India. The system includes a curated dataset containing information about 36 travel destinations categorized according to travel type, climate conditions, estimated costs, and tourist attractions.

The project encompasses multiple components including:

- Machine learning model development and training.
- Backend development using Django.
- Frontend interface design and responsive templating.
- Database schema creation and management.
- User authentication and account management.
- Dynamic itinerary generation.
- Saved trip management with soft-delete functionality.
- Travel recommendation and destination filtering systems.

The application functions as a predictive recommendation and planning engine rather than a complete travel booking platform. Therefore, the current implementation does not integrate real-time third-party APIs for hotel reservations, transportation booking, payment gateways, or live weather forecasting. Instead, the system focuses on demonstrating how machine learning and web technologies can be integrated to automate and personalize travel planning processes.

Although the project currently focuses on Indian tourism destinations, the architecture is scalable and can be expanded to support international destinations and real-time travel services in future versions. The modular system design allows easy integration of

additional features such as hotel booking APIs, chatbot assistance, multilingual support, collaborative travel planning, and advanced recommendation algorithms.

Chapter 2: Literature Review

2.1 Traditional Travel Planning

Travel planning has evolved significantly over the years, transitioning from entirely manual processes to digitally assisted systems powered by intelligent technologies. In the traditional tourism model, travelers relied heavily on physical travel agencies and human travel consultants to organize vacations, transportation, accommodation, and sightseeing activities. Travel agents manually analyzed customer requirements such as destination preferences, travel duration, budget limitations, and seasonal conditions before recommending suitable travel packages. These services were often personalized because human agents could directly communicate with customers and understand their expectations in detail.

However, traditional travel planning methods suffered from several limitations. The process was time-consuming, expensive, and highly dependent on the expertise and experience of travel agents. Since recommendations were manually generated, they were susceptible to human error, personal bias, and limited information availability. Travel agencies also relied on static tour packages that lacked flexibility and customization. Customers often had to adjust their preferences according to pre-designed itineraries rather than receiving plans specifically tailored to their needs.

The introduction of the internet and web technologies transformed the tourism industry dramatically. During the era of Web 1.0 and later Web 2.0, online travel platforms emerged as alternatives to physical travel agencies. Websites such as Tripadvisor, MakeMyTrip, and other online booking systems digitized travel-related services including hotel reservations, transportation booking, destination reviews, and tourism information. These platforms increased accessibility and convenience by allowing users to search for travel options directly from their devices.

Although digital travel platforms simplified booking procedures, they introduced new challenges. Users were required to manually browse through extensive lists of destinations, accommodations, and reviews before making decisions. The abundance of information often caused decision fatigue and information overload. While these systems provided search and filtering functionalities, they still relied heavily on user effort for itinerary planning and destination selection. In most cases, travelers had to independently compare costs, weather conditions, tourist attractions, and travel schedules to create suitable itineraries.

Furthermore, traditional digital tourism platforms generally focused on transactional services such as booking flights and hotels rather than providing intelligent planning assistance. They lacked the capability to understand complex user constraints and generate personalized travel recommendations automatically. As tourism demand continued to increase, researchers and developers recognized the need for intelligent systems capable of automating travel planning processes and reducing user workload through artificial intelligence and recommendation technologies.

2.2 Recommender Systems in Tourism

Recommender systems have become one of the most influential applications of artificial intelligence in modern digital platforms. These systems are widely used in e-commerce, entertainment, social media, and online marketplaces to analyze user preferences and provide personalized suggestions. Companies such as Amazon and Netflix have successfully utilized recommendation systems to improve customer engagement, user satisfaction, and service efficiency.

In the tourism industry, recommender systems play an important role in helping travelers discover destinations, accommodations, restaurants, and travel activities. The growing complexity of tourism-related information has made intelligent recommendation technologies essential for simplifying user decision-making processes. Tourism recommender systems attempt to analyze user behavior, preferences, historical travel patterns, ratings, and contextual information to generate personalized travel suggestions.

Several recommendation techniques are commonly applied within tourism applications. Among the most widely used approaches are Content-Based Filtering and Collaborative Filtering.

Content-Based Filtering

Content-Based Filtering recommends items based on similarities between user preferences and item attributes. In tourism applications, this method analyzes destination characteristics such as climate, travel type, activities, cost range, and cultural features. If a user previously showed interest in beach destinations or adventure tourism, the system recommends destinations with similar attributes.

The main advantage of Content-Based Filtering is its ability to generate personalized recommendations without relying on other users' data. However, the method often suffers from limited diversity because recommendations are restricted to items similar to those previously selected by the user. It may also struggle to recommend entirely new or unexplored destinations.

Collaborative Filtering

Collaborative Filtering generates recommendations based on similarities between users. The system identifies travelers with comparable interests and suggests destinations preferred by similar users. This technique is widely used because it can identify hidden patterns and relationships within large datasets.

Despite its popularity, Collaborative Filtering faces several limitations in tourism systems. It often suffers from the "cold start" problem, where recommendations become inaccurate when insufficient user data is available. In addition, tourism decisions are influenced by dynamic contextual factors such as weather, budget, travel duration, and seasonal conditions, which are difficult to capture using collaborative approaches alone.

Hybrid Recommendation Approaches

To overcome the limitations of individual recommendation methods, modern tourism systems increasingly adopt hybrid recommendation models that combine multiple techniques. Hybrid systems integrate content-based analysis, collaborative filtering,

contextual information, and machine learning algorithms to improve recommendation accuracy and personalization.

In travel planning, users often specify both hard constraints and soft preferences. Hard constraints include fixed limitations such as budget, available travel days, and climate requirements. Soft preferences refer to subjective interests such as relaxation, adventure, cultural exploration, or family-friendly experiences. Traditional recommendation systems are not always effective at simultaneously processing these complex parameters.

As a result, supervised machine learning models and intelligent classification algorithms are gaining popularity in tourism recommendation research. These models treat destination recommendation as a predictive classification problem, enabling systems to generate more accurate and constraint-aware travel suggestions. However, fully integrated AI-powered travel planning systems capable of recommendation, itinerary generation, and personalized trip management within a single platform remain relatively limited in existing research and commercial deployment.

2.3 Machine Learning Applications in Travel

Machine learning has emerged as one of the most powerful technologies in modern intelligent systems, enabling applications to automatically learn patterns from data and improve decision-making capabilities without explicit programming. In the travel and tourism industry, machine learning techniques are increasingly used to enhance customer experiences, optimize travel recommendations, predict user behavior, and automate itinerary planning processes.

The growing availability of travel datasets, user interaction data, and destination-related information has enabled researchers to apply predictive analytics and classification models within tourism systems. Machine learning algorithms can analyze relationships between user constraints and travel destinations to generate highly personalized recommendations with improved speed and accuracy.

Several machine learning algorithms have been widely explored for tourism recommendation tasks:

Decision Tree Algorithms

Decision Trees are supervised learning models that classify data based on decision rules derived from input features. In travel recommendation systems, Decision Trees can analyze parameters such as budget, trip duration, climate preference, and travel category to identify suitable destinations. The algorithm is easy to interpret and provides clear decision-making logic, making it suitable for tourism applications requiring explainable recommendations.

Random Forest Algorithms

Random Forest is an ensemble learning method that combines multiple Decision Trees to improve prediction accuracy and reduce overfitting. This algorithm is particularly effective for handling diverse travel datasets with multiple input variables. Random Forest models can generate robust and reliable destination recommendations by aggregating predictions from multiple classifiers.

Support Vector Machines (SVM)

Support Vector Machines are classification algorithms designed to identify optimal boundaries between data categories. In tourism systems, SVM models can classify destinations according to travel preferences and user constraints. Although SVM algorithms often provide high classification accuracy, they may require more computational resources and careful parameter tuning for large datasets.

K-Nearest Neighbor (KNN)

The K-Nearest Neighbor algorithm identifies similar data points based on feature proximity. In travel recommendation systems, KNN can suggest destinations by comparing current user inputs with previously categorized travel data. The simplicity of KNN makes it useful for smaller datasets and prototype-level systems.

Neural Networks and Deep Learning

Recent advancements in artificial intelligence have introduced deep learning techniques into tourism applications. Neural networks can process complex relationships between

user behavior, contextual information, and destination characteristics. These models are capable of generating highly adaptive and personalized travel recommendations. However, deep learning systems generally require large datasets, high computational power, and more advanced training procedures.

In many modern travel recommendation systems, destination prediction is treated as a multi-class classification problem. Input variables such as travel budget, number of travel days, climate preference, and travel type are processed as feature vectors, while travel destinations represent output classes. Machine learning models learn patterns between these features and destination categories to predict the most suitable travel option for each user.

The integration of machine learning into tourism systems offers several advantages:

- Improved personalization and recommendation accuracy.
- Faster decision-making and itinerary generation.
- Reduced manual effort in travel planning.
- Better handling of complex user constraints.
- Scalable and adaptive recommendation capabilities.

Despite these advancements, many existing travel recommendation platforms still focus on isolated functionalities such as hotel suggestions or destination popularity rankings. Comprehensive AI-based systems that combine machine learning recommendations, itinerary generation, user authentication, and trip management into a unified web application remain relatively limited.

The proposed project contributes to this area by integrating machine learning-based destination prediction with dynamic itinerary generation and secure web-based travel management using the Django framework and Scikit-learn machine learning technologies. The system demonstrates how artificial intelligence can be effectively utilized to simplify and personalize the travel planning process while improving user experience and operational efficiency.

Chapter 3: System Analysis and Design

This chapter presents the overall system analysis, architectural design, workflow mechanisms, and technological foundations of the AI-Based Trip Planner Web Application. The system is designed using a modular, scalable, and maintainable architecture that combines modern web development principles with machine learning technologies. The architecture integrates a web-based frontend, a Django-powered backend server, a machine learning recommendation engine, and a relational database management system to provide personalized trip recommendations and dynamic itinerary generation.

The chapter also explains the internal communication between system components, database structures, data flow mechanisms, and the technologies used during implementation. Diagrams and architectural representations are included to provide a clear understanding of the overall system functionality and interactions.

3.1 System Architecture

The AI-Based Trip Planner Web Application follows the Model-View-Template (MVT) architectural pattern provided by the Django framework. The system architecture is designed to separate business logic, user interface rendering, machine learning operations, and database interactions into independent layers. This modular design improves maintainability, scalability, security, and performance.

The architecture consists of the following major components:

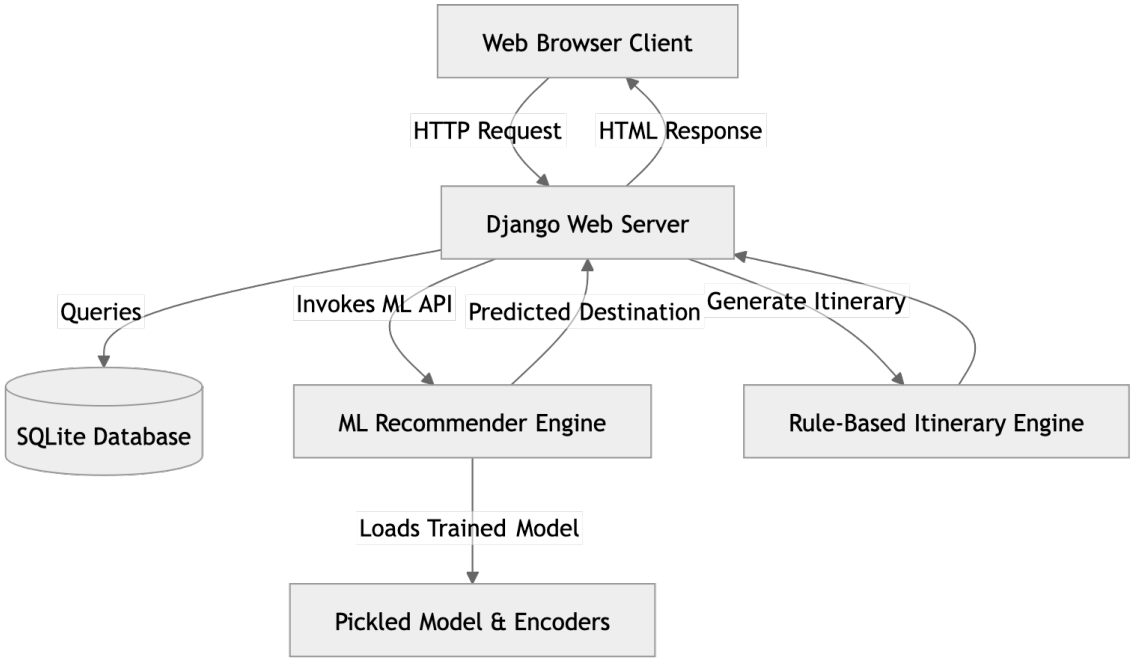
- Web Browser Client
- Django Web Server & SQLite Database
- Machine Learning Recommendation Engine
- Rule-Based Itinerary Generation Engine
- Serialized Machine Learning Models

The system workflow begins when the user accesses the web application through a browser interface and submits travel preferences such as budget, number of travel days, preferred climate, and travel type. The Django backend receives these inputs, validates the data, and forwards the processed features to the machine learning recommendation engine.

The machine learning module loads pre-trained models and label encoders from serialized .pkl files and predicts the most suitable travel destination based on user constraints. Once the prediction is generated, the backend invokes the itinerary generation engine, which dynamically creates a day-by-day travel plan including activities, estimated costs, and destination guidance.

The generated itinerary and associated metadata are then stored in the SQLite database and returned to the frontend for rendering. Finally, the user receives a complete personalized travel plan through an interactive web interface.

Figure 3.1: High-Level System Architecture



Explanation of Architectural Components

1. Web Browser Client

The client layer represents the user interface through which travelers interact with the application. It is developed using HTML5, CSS3, and JavaScript to provide responsive and interactive user experiences. The frontend allows users to:

- Register and log in securely
- Enter travel constraints
- Request personalized recommendations
- View generated itineraries
- Save and manage trips
- Access previous travel plans

The browser communicates with the Django backend through HTTP requests and receives rendered HTML or JSON responses.

2. Django Web Server

The Django server acts as the central processing unit of the application. It handles routing, request processing, business logic, authentication, database communication, and API interactions.

Key responsibilities include:

- Managing user authentication and sessions
- Processing trip recommendation requests
- Communicating with the ML engine
- Generating dynamic itineraries
- Storing and retrieving trip records
- Rendering frontend templates

The Django framework provides built-in security features such as CSRF protection, password hashing, and ORM-based database handling.

3. Machine Learning Recommendation Engine

The recommendation engine is the intelligent core of the application. It receives processed user inputs and predicts suitable travel destinations using a trained Decision Tree classification model.

The engine performs the following tasks:

- Loading serialized models and encoders
- Encoding categorical features
- Running prediction inference
- Returning destination recommendations

The ML module is implemented using Scikit-learn and operates independently from the frontend.

4. SQLite Database

The system uses SQLite3 as the primary relational database for storing application data. The database stores:

- User accounts
- Saved trips
- Itinerary information
- Travel metadata
- Soft-deleted records

SQLite is lightweight, easy to configure, and suitable for prototype-level deployments and academic research projects.

5. Rule-Based Itinerary Engine

The itinerary engine generates realistic day-by-day travel schedules after destination prediction. It uses rule-based logic to allocate attractions, activities, and estimated budgets according to trip duration and travel category.

Generated itinerary details include:

- Daily activities
- Tourist attractions
- Estimated travel costs
- Accommodation suggestions
- Local travel guidance

3.2 Data Flow and Sequence Diagrams

The data flow mechanism defines how information moves across different modules of the system during travel recommendation generation. The sequence diagram below illustrates the interaction between the user, frontend interface, Django backend, machine learning engine, and database.

When a user submits trip preferences through the frontend form, the backend processes the request and sends feature vectors to the machine learning model. After destination prediction, the itinerary engine creates a detailed travel schedule, and the resulting data is stored in the database before being displayed to the user.

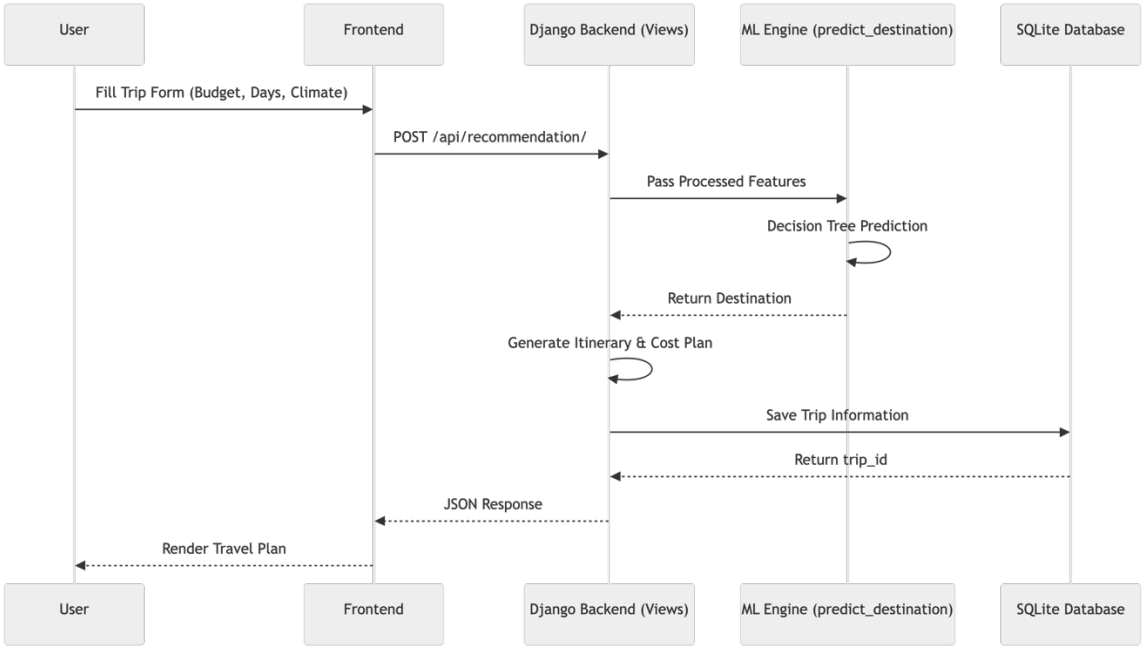


Figure 3.2: User Interaction Sequence Diagram

Detailed Workflow Explanation

Step 1: User Input Collection

The user fills out a travel planning form containing:

- Budget range
- Number of travel days
- Preferred climate
- Travel category

The frontend validates the form before sending the request to the backend.

Step 2: API Request Submission

The frontend sends a POST request to the Django API endpoint responsible for travel recommendations.

Example endpoint:

`/api/recommendation/`

The request payload contains user constraints encoded as JSON data.

Step 3: Machine Learning Inference

The Django backend preprocesses the data and passes feature vectors to the machine learning model.

The ML engine performs:

- Label encoding
- Feature transformation
- Decision Tree inference

The engine predicts the most suitable destination and returns the result to the backend.

Step 4: Dynamic Itinerary Generation

Once the destination is predicted, the backend invokes the itinerary generation engine. The engine creates a structured travel schedule based on:

- Number of days
- Destination category
- Estimated costs
- Available tourist attractions

Step 5: Database Storage

The generated trip plan is stored in the SQLite database using Django ORM. The system supports:

- Saved trips
- Soft-delete functionality
- User-specific travel history

Step 6: Response Rendering

The final itinerary and recommendation data are returned to the frontend, where the results page dynamically displays the travel plan to the user.

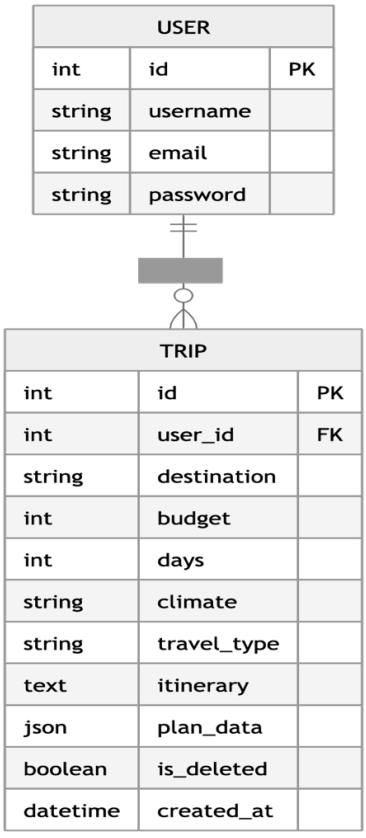
3.3 Database Schema and ER Diagram

The application database is designed using relational database principles to ensure consistency, maintainability, and scalability. The primary entities in the system are:

- User
- Trip

Each user can create multiple trip plans, establishing a one-to-many relationship between the User and Trip entities.

Figure 3.3: Entity Relationship Diagram (ERD)



Database Entity Description

USER Entity

The USER table stores authentication and account-related information.

Attributes include:

- id: Unique primary key
- username: User login identifier
- email: Registered email address
- password: Encrypted user password

TRIP Entity

The TRIP table stores all generated travel plans and itinerary information.

Attributes include:

- destination: Predicted destination
- budget: User budget
- days: Trip duration
- climate: Preferred climate
- travel_type: Selected travel category
- itinerary: Generated day-wise schedule
- plan_data: JSON-formatted structured trip data
- is_deleted: Soft-delete status
- created_at: Timestamp of trip creation

3.4 Technology Stack

The AI-Based Trip Planner Web Application utilizes a combination of backend frameworks, machine learning libraries, frontend technologies, and database systems to provide a scalable and intelligent travel planning platform.

Table 3.4: Technology Stack Breakdown

Component	Technology	Purpose
Backend	Python, Django	Web framework, routing, ORM handling
Machine Learning	Scikit-Learn, Pandas	Data processing and prediction model
Database	SQLite3	Persistent data storage
Frontend	HTML5, CSS3, JavaScript	User interface and rendering
API Framework	Django REST Framework	JSON APIs and serialization
Model Serialization	Joblib	Saving trained ML models
Version Control	Git	Source code management

Backend Technologies

The backend is implemented using Python and Django. Django provides secure authentication, database abstraction through ORM, URL routing, middleware support, and template rendering functionalities.

Advantages include:

- Rapid development
- Built-in security

- Scalable architecture
- Easy database integration

Machine Learning Technologies

The machine learning module uses:

- Scikit-learn
- Pandas
- NumPy

These libraries support:

- Data preprocessing
- Label encoding
- Decision Tree training
- Prediction inference

Chapter 4: Machine Learning Implementation

The core intelligence of the AI-Based Trip Planner Web Application resides in its machine learning recommendation engine. This chapter explains the dataset generation process, preprocessing techniques, feature engineering, classification algorithms, model training procedures, and serialization mechanisms used to build the predictive recommendation system.

The machine learning pipeline is responsible for transforming user travel constraints into personalized destination recommendations with high speed and efficiency.

4.1 Dataset Generation and Preprocessing

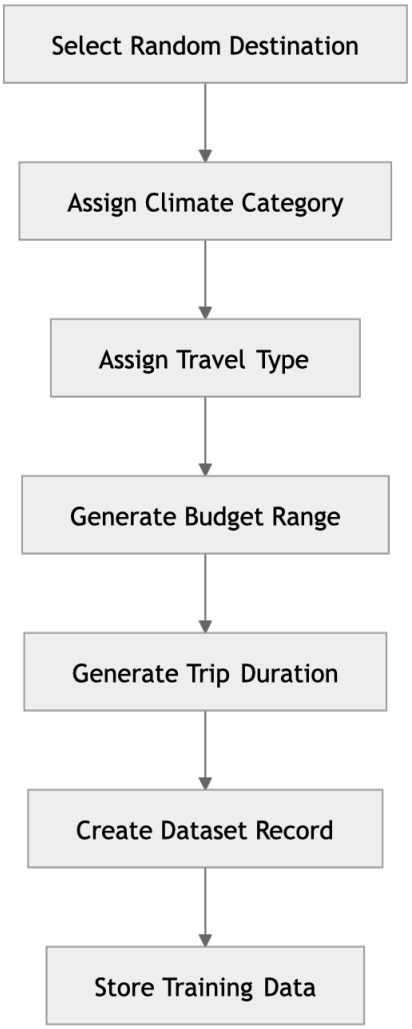
One of the major challenges in travel recommendation research is the lack of publicly available datasets containing structured mappings between travel constraints and personalized destinations. Since real-world tourism datasets are often incomplete, inconsistent, or commercially restricted, this project utilizes a carefully designed synthetic dataset generation approach.

The system includes a curated list of 36 Indian travel destinations categorized according to:

- Climate type
- Travel category
- Budget range
- Tourist attractions
- Travel suitability

To simulate realistic user behavior, the application generates approximately 5,000 synthetic travel records.

Synthetic Dataset Generation Workflow



Dataset Features

Each generated dataset record contains:

Feature	Description
Budget	Estimated travel cost
Days	Trip duration
Climate	Preferred climate type
Travel Type	Adventure, Family, etc.
Destination	Target prediction label

4.2 Feature Engineering and Encoding

Machine learning algorithms require numerical input values. Therefore, categorical features such as climate type and travel category must be converted into numeric representations before model training.

The project uses LabelEncoder from Scikit-learn to encode categorical variables.

Table 4.1: Feature Encoding Mapping

Feature	Original Values	Encoded Values
Climate	Hot, Cold, Moderate	0, 1, 2
Travel Type	Adventure, Cultural, Family, Relaxation	0, 1, 2, 3
Destination	36 destination names	0 to 35

Figure 4.1: Feature Transformation Pipeline

4.3 Decision Tree Classifier Algorithm

The system utilizes the DecisionTreeClassifier algorithm with a maximum depth of 8.

Decision Trees are non-parametric supervised learning algorithms that recursively split datasets into branches based on feature conditions. They are particularly suitable for travel recommendation systems because they can effectively handle both categorical and numerical variables while producing interpretable decision rules.

Advantages of Decision Trees

- Easy to interpret
- Fast inference speed
- Handles categorical features efficiently
- Requires minimal preprocessing
- Suitable for multi-class classification

Mathematical Basis: Gini Impurity

The Decision Tree uses the Gini Impurity metric to determine the optimal feature split during training.

$\text{Gini}(p) = 1 - \sum_{i=1}^{j} p_i^2$

Where:

- (p_i) represents the probability of an item belonging to class (i)
- (J) represents the total number of classes

Lower Gini impurity values indicate better node purity and more accurate classification splits.

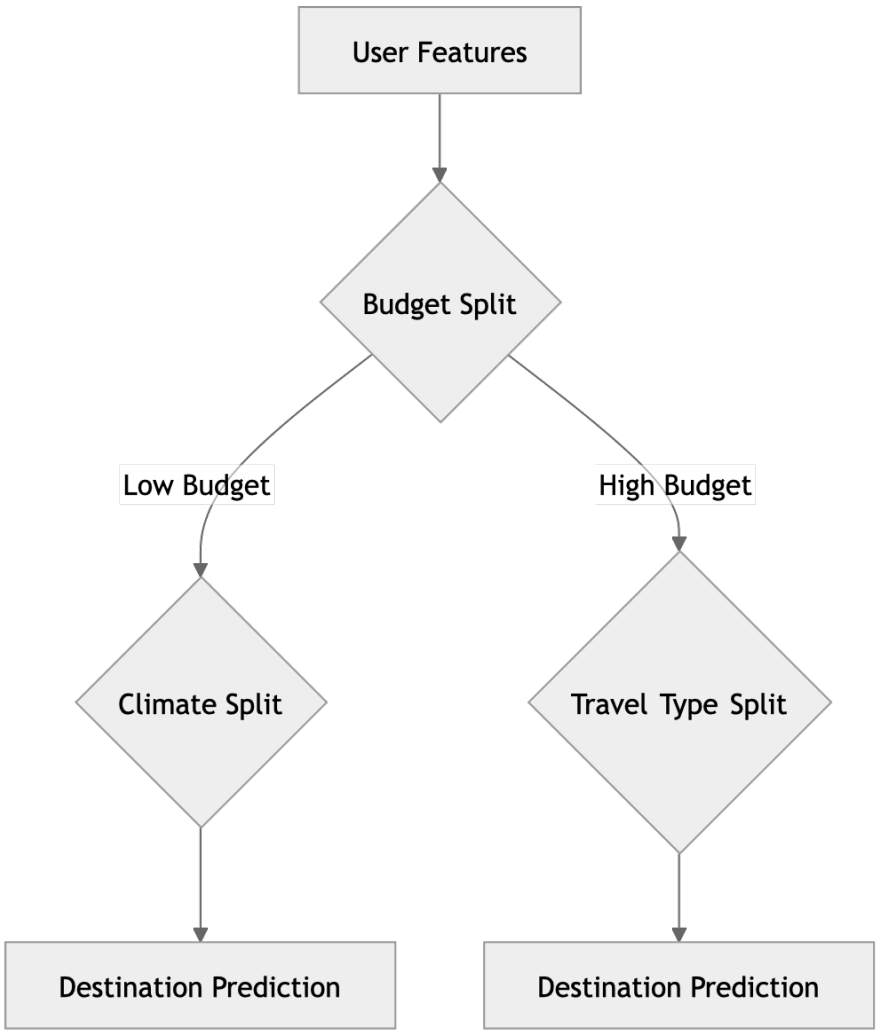


Figure 4.2: Decision Tree Workflow

4.4 Model Training and Serialization

After preprocessing and feature encoding, the dataset is divided into training and testing subsets. The model is trained using the .fit(X, y) method provided by Scikit-Learn.

The trained Decision Tree model learns patterns between travel constraints and destination categories to generate accurate recommendations during inference.

Figure 4.3: Training Pipeline

Model Serialization

Once training is completed, the trained model and encoders are serialized using the Joblib library. Serialization enables the application to reuse trained models without retraining during runtime.

Serialized files include:

- *trip_model.pkl*
- *climate_encoder.pkl*
- *travel_encoder.pkl*
- *destination_encoder.pkl*

These files are loaded by the Django backend during application startup, enabling fast real-time predictions and efficient deployment.

Benefits of Serialization

- Faster application startup
- Reduced computational overhead
- Real-time recommendation support
- Easier deployment and portability
- Separation of training and inference environments

Chapter 5: Web Application Development

This chapter discusses the complete development process of the AI-Based Trip Planner Web Application. It explains the backend implementation using the Django framework, the integration of machine learning components, authentication systems, API development, itinerary generation logic, and frontend interaction mechanisms. The chapter also highlights how the application architecture ensures scalability, maintainability, security, and efficient user interaction.

The system is designed using modular development principles, where each component performs a dedicated function while interacting seamlessly with other modules. This approach allows the application to support intelligent travel recommendations, dynamic itinerary generation, secure user management, and responsive user interfaces within a unified ecosystem.

5.1 Django Framework and MVT Architecture

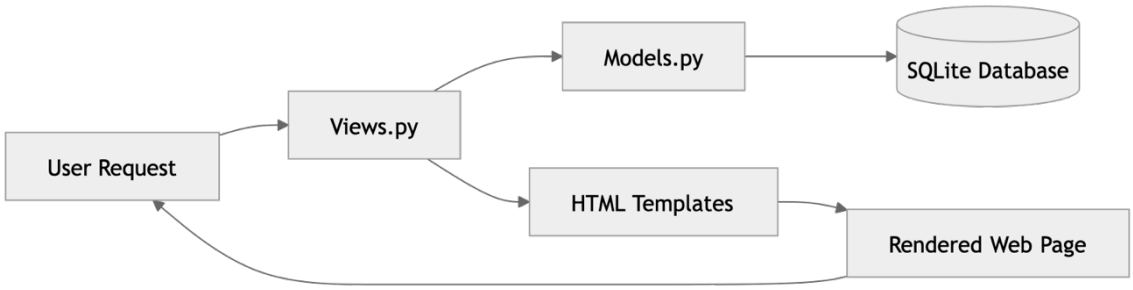
The AI-Based Trip Planner Web Application is developed using the Django framework, one of the most widely used Python-based web development frameworks. Django follows the Model-View-Template (MVT) architectural pattern, which separates the application into independent layers for data handling, business logic, and presentation rendering.

The MVT architecture improves:

- Code organization
- Scalability
- Maintainability
- Security
- Reusability

By separating concerns into dedicated modules, the application becomes easier to develop, debug, and extend with additional features in future versions.

Figure 5.1: Django MVT Architecture



5.1.1 Models Layer

The Models layer represents the database structure and data management logic of the application. Models are implemented in:

trips/models.py

The primary model in the system is the Trip model, which stores travel recommendations, itineraries, and user-specific travel metadata.

The model utilizes advanced Django ORM features including:

- JSONField
- Foreign key relationships
- Timestamp management
- Soft-delete functionality

The use of JSONField allows the application to efficiently store structured itinerary data such as:

- Daily activities
- Estimated expenses
- Tourist attractions
- Transportation details
- Accommodation information

This approach provides flexibility when handling dynamic itinerary structures that may vary across destinations and trip durations.

Example Trip Model Structure

```
class Trip(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    destination = models.CharField(max_length=100)
    budget = models.IntegerField()
    days = models.IntegerField()
    climate = models.CharField(max_length=50)
    travel_type = models.CharField(max_length=50)
    itinerary = models.TextField()
    plan_data = models.JSONField()
    is_deleted = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)
```

5.1.2 Views Layer

The Views layer contains the business logic and request-handling mechanisms of the application. Views are implemented in:

trips/views.py

Views act as intermediaries between the frontend interface, database models, and machine learning modules.

Responsibilities of the Views layer include:

- Processing HTTP requests
- Validating form data
- Calling machine learning functions
- Generating itineraries
- Rendering HTML templates
- Returning JSON responses

The views interact with the recommendation engine to predict destinations and generate travel plans dynamically.

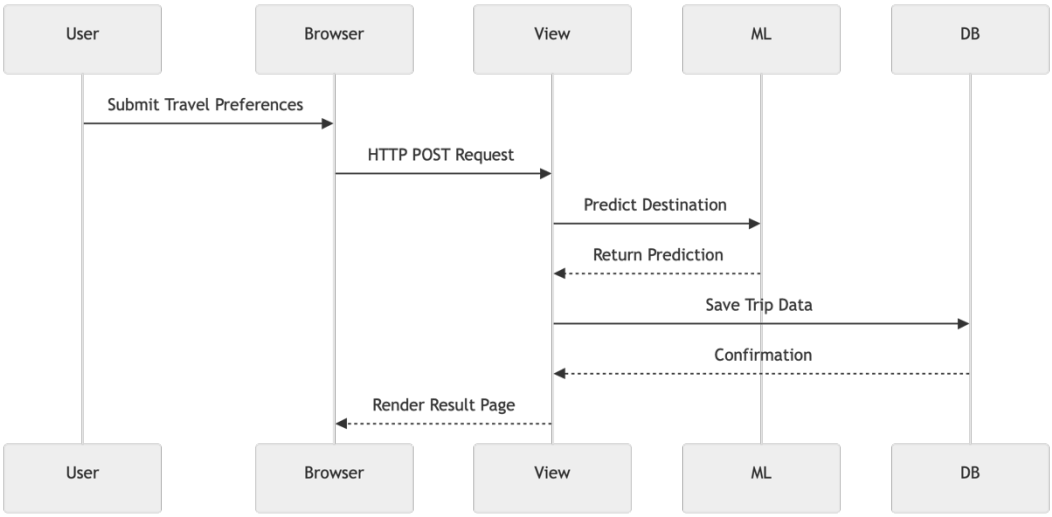


Figure 5.2: Request Handling Workflow

5.1.3 Templates Layer

The Templates layer manages the frontend rendering and user interface presentation.

The application uses:

- HTML5
- CSS3
- JavaScript
- Django Template Language (DTL)

Templates dynamically display user-specific data using Django template tags such as:

```
{% block content %}
{{ destination }}
{{ itinerary }}
```

The template system allows reusable layouts and modular frontend design. Components such as navigation bars, dashboards, footers, and travel cards are shared across multiple pages using template inheritance.

5.2 User Authentication and Session Management

Security and personalized user experience are essential components of the application. The system uses Django's built-in authentication framework to securely manage user accounts, sessions, and permissions.

The authentication system enables users to:

- Register accounts
- Log in securely
- Maintain active sessions
- Save personalized trips
- Access dashboards
- Restore deleted itineraries

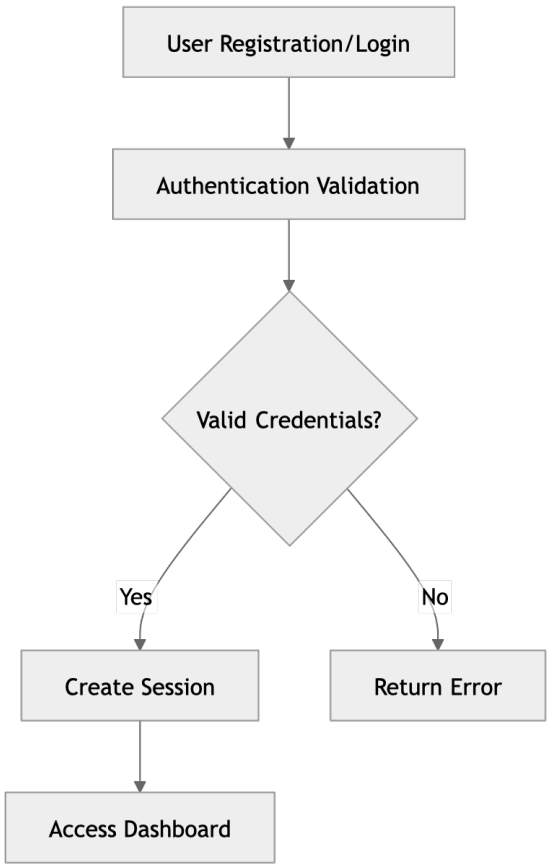


Figure 5.3: Authentication Workflow

5.2.1 User Registration

Custom registration views allow users to create secure accounts using usernames, email addresses, and passwords.

Registration functions include:

- Form validation
- Duplicate account prevention
- Password hashing
- User creation

Passwords are securely encrypted using Django's built-in hashing algorithms before storage in the database.

5.2.2 User Login and Logout

The application includes custom authentication views such as:

```
login_page()
register_page()
logout_page()
```

These views manage:

- Session cookies
- Authentication tokens
- Secure redirection
- Login state persistence

When users log in successfully, Django automatically creates a session object that remains active until logout or expiration.

5.2.3 Access Control and Route Protection

Protected application routes such as dashboards and itinerary management pages use the @login_required decorator.

Example:

```
@login_required
def dashboard(request):
    ...
```

This mechanism ensures that only authenticated users can access sensitive travel data and personalized recommendations.

5.3 Dynamic Itinerary Generation Module

One of the most important components of the application is the Dynamic Itinerary Generation Module. While the machine learning model predicts only the destination name, the itinerary engine transforms that prediction into a complete travel experience.

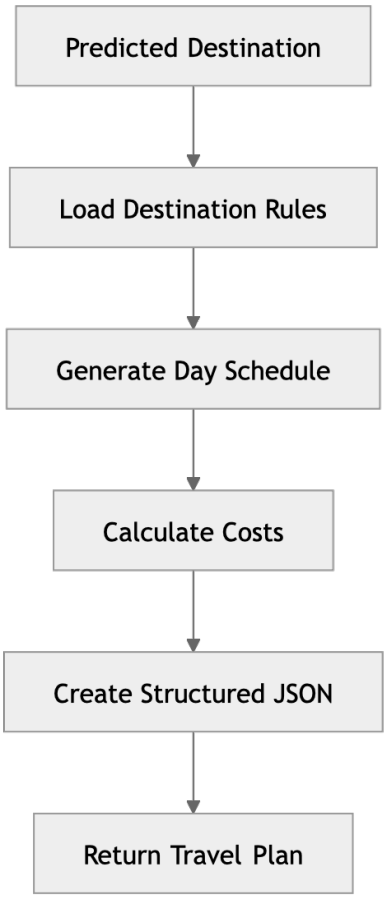
For example:

Predicted Destination: Manali

The itinerary module expands this output into:

- Day-by-day schedules
- Cost estimation
- Attraction recommendations
- Activity planning
- Accommodation suggestions

Figure 5.4: Itinerary Generation Pipeline



5.3.1 Cost Estimation Logic

The system calculates trip costs dynamically based on:

- Destination
- Number of days
- Number of travelers
- Travel category
- Activity intensity

The engine includes predefined destination cost structures such as:

- Accommodation costs
- Food expenses
- Transportation costs
- Activity fees

Adventure trips may increase activity costs using multipliers.

Example:

Adventure Activity Cost = Base Cost × 1.5

This enables realistic and adaptive budget estimation.

5.3.2 Day-by-Day Scheduling

The application contains a predefined itinerary dictionary called:

generate_itinerary

This structure stores destination-specific schedules containing:

- Tourist attractions
- Daily activities
- Recommended timings
- Local travel suggestions

The itinerary engine automatically:

- Truncates schedules for shorter trips
- Extends schedules for longer trips
- Rearranges activities dynamically

This ensures flexible travel planning according to user-selected trip durations.

Example Itinerary Structure

```
{
  "Day 1": "Arrival and local sightseeing",
  "Day 2": "Adventure activities and trekking",
  "Day 3": "Cultural exploration and shopping"
}
```

5.4 API Integration and Django REST Framework

The application uses Django REST framework for API development and JSON serialization. REST APIs enable communication between frontend interfaces and backend services while supporting scalable application design.

The recommendation API is implemented using:

TripRecommendationView

which inherits from:

APIView

Figure 5.5: API Communication Architecture

5.4.1 Request Validation

The application uses serializers such as:

TripRequestSerializer

to validate incoming request data.

Validation checks include:

- Required fields
- Numeric constraints
- Allowed climate values
- Travel category validation

This prevents invalid or malicious data from reaching the backend logic.

5.4.2 JSON Response Structure

The API returns structured JSON responses containing:

- Predicted destination
- Budget breakdown
- Estimated expenses
- Daily itinerary
- Travel metadata

Example response:


```
{
 "destination": "Manali",
 "budget": 25000,
 "days": 5,
 "itinerary": {
   "Day 1": "Arrival and sightseeing",
   "Day 2": "Adventure sports"
 }
```


Table 5.1: API Endpoint Specifications

Endpoint	Method	Payload	Response
/api/recommendation/	POST	{budget, days, climate, travel_type}	JSON itinerary and cost breakdown

Chapter 6: User Interface and Experience

The success of modern web applications depends heavily on usability, responsiveness, and visual appeal. This chapter explains the frontend architecture, UI/UX principles, interactive components, and user experience strategies used in the AI-Based Trip Planner Web Application.

The frontend is designed to provide users with a seamless and engaging travel planning experience while maintaining simplicity and accessibility.

6.1 Frontend Architecture

The frontend architecture focuses on responsive design, smooth navigation, and visually attractive layouts. Although the application uses traditional server-side rendering through Django templates, the interface is designed to simulate modern single-page application behavior.

Frontend technologies include:

- HTML5
- CSS3
- JavaScript
- Django Template Language

Figure 6.1: Frontend Rendering Workflow

Responsive Design Features

The application supports:

- Desktop devices
- Tablets
- Smartphones

Responsive layouts are achieved through:

- Flexible grid systems
- Adaptive image scaling
- Media queries
- Mobile-friendly navigation

Dynamic Destination Images

To improve visual appeal, the application integrates fallback destination images using:

Unsplash Source API

If local images are unavailable, the system dynamically loads travel-related images based on destination names.

This feature enhances:

- Visual engagement
- User immersion
- Destination attractiveness

6.2 Key Application Interfaces

The application contains multiple interactive interfaces designed for different stages of the travel planning process.

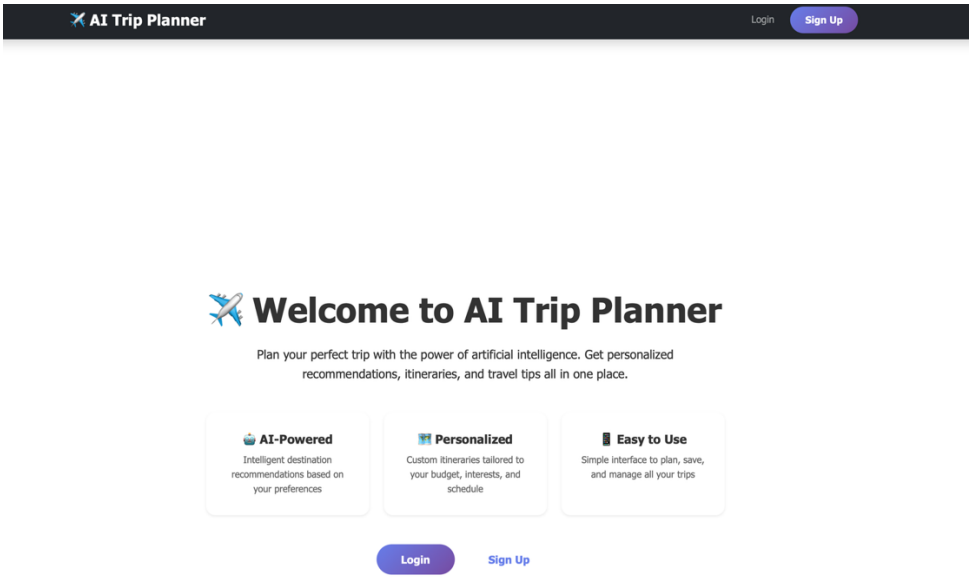
6.2.1 Landing and Authentication Pages

The landing page acts as the entry point of the application and focuses on:

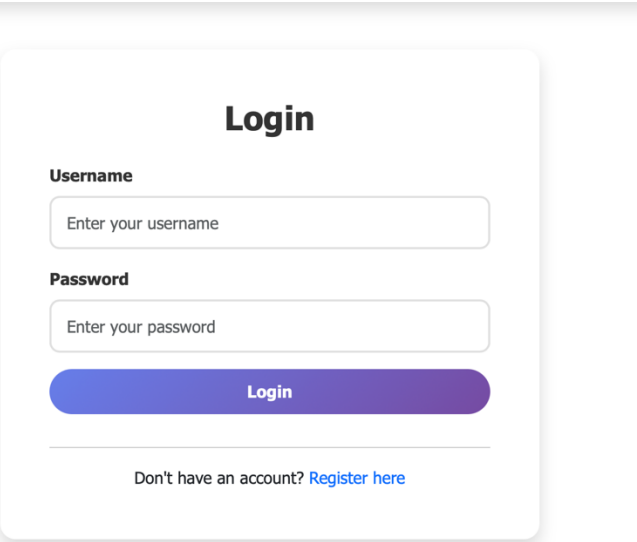
- User engagement
- Travel inspiration
- Easy navigation
- Quick registration access

Authentication pages include:

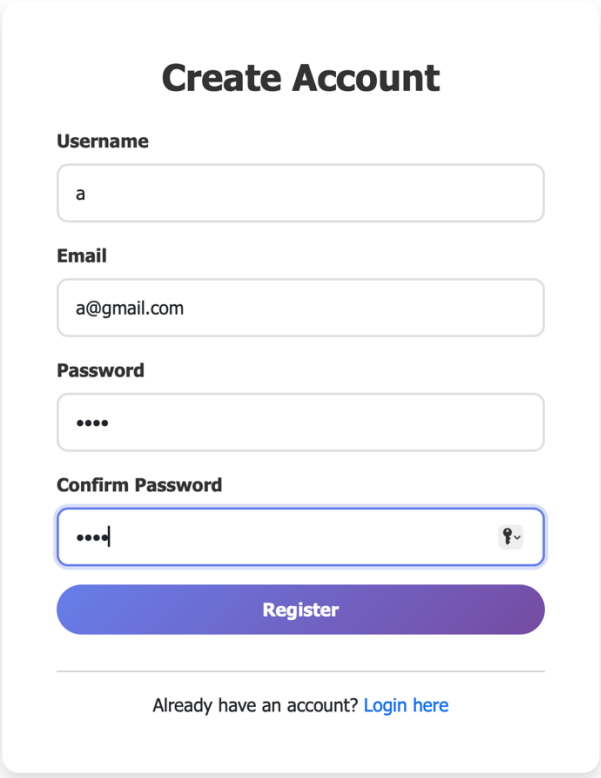
- Login forms
- Registration forms
- Password validation
- Session management



Home page



Login page



Registrastion Page

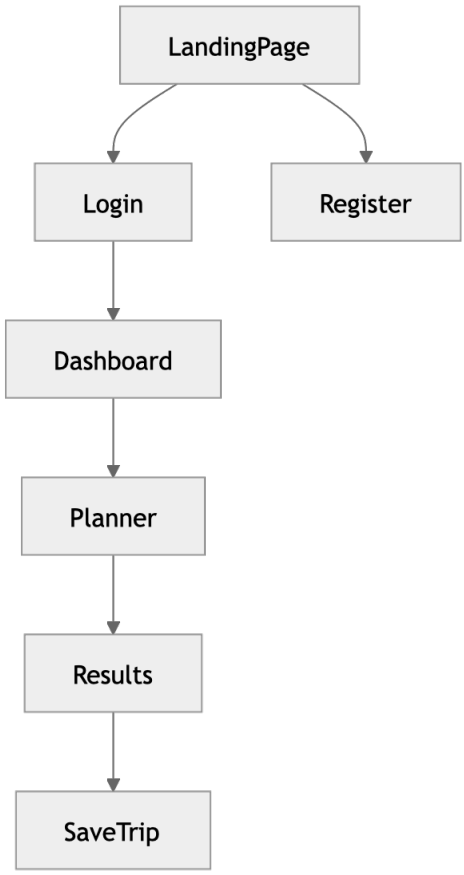


Figure 6.2: User Navigation Structure

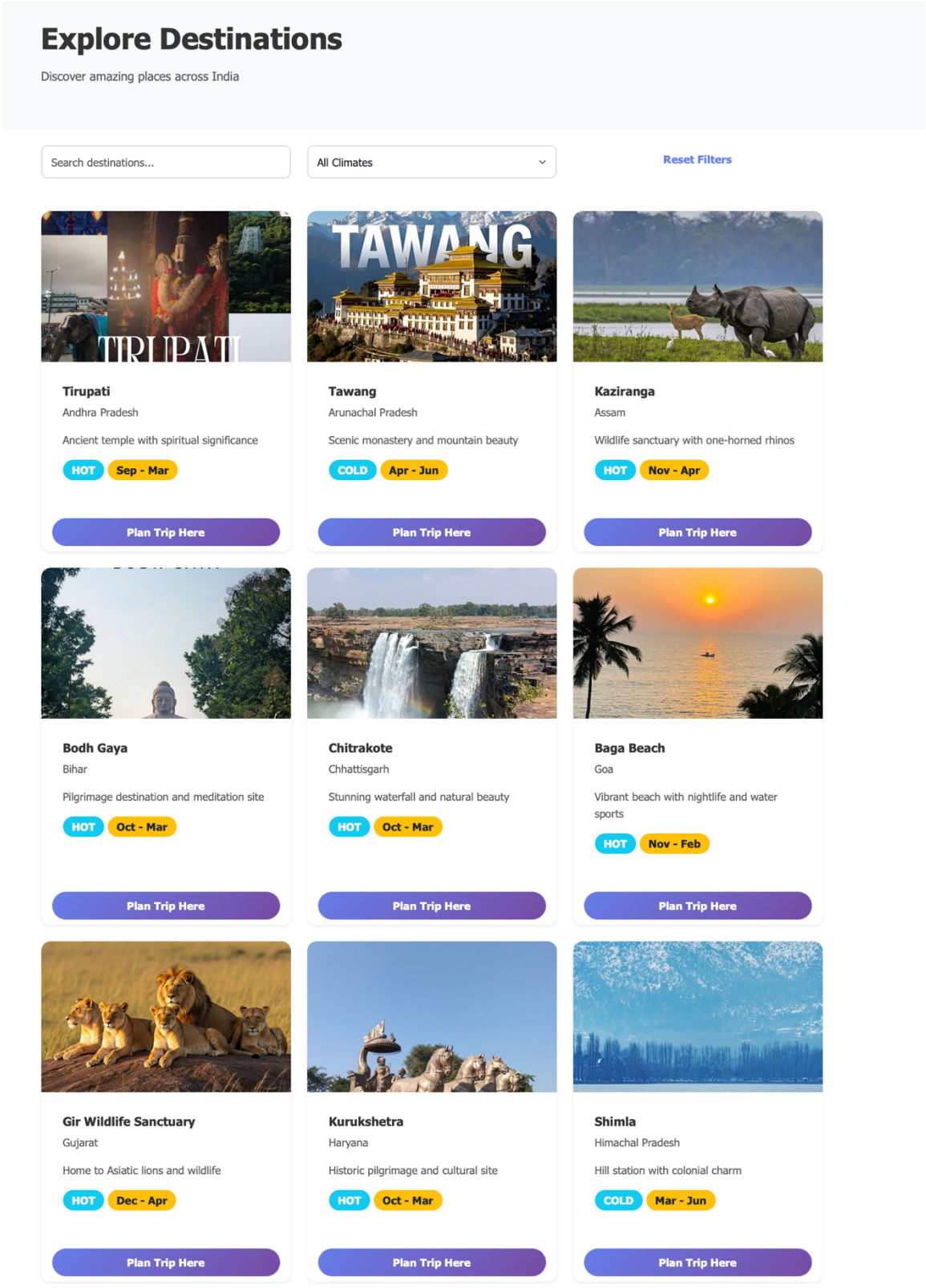
6.2.2 Destinations Explorer

The destination explorer displays all 36 travel destinations using responsive grid layouts.

Each destination card contains:

- Destination image
- Climate information
- Estimated budget
- Travel category
- Short description

This interface helps users explore travel options before requesting recommendations.



Destinations Explorer

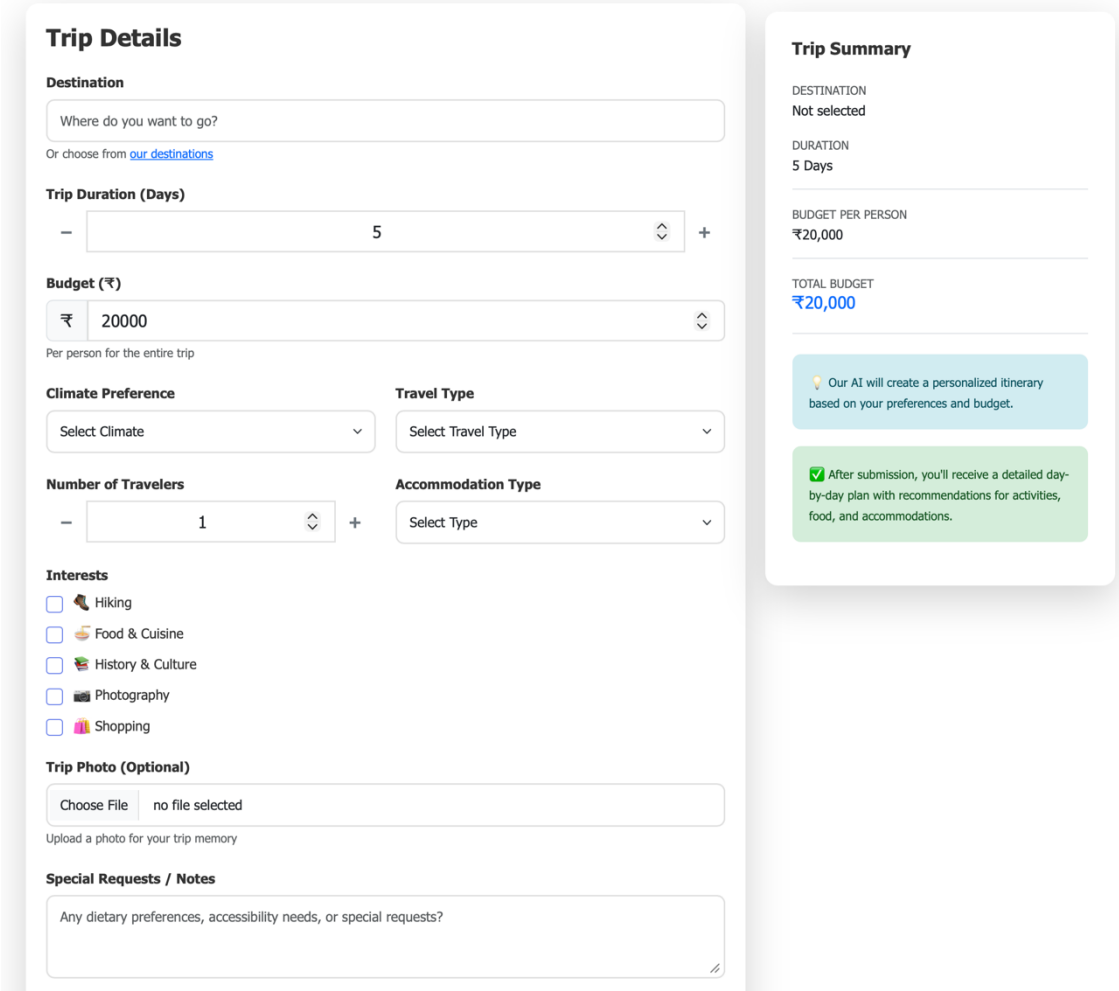
6.2.3 Trip Planning Form

The trip planning form acts as the primary interaction point for recommendation generation.

The form securely collects:

- Budget
- Travel days
- Climate preference
- Travel type

Interactive dropdowns and validation improve user input accuracy and usability.



Trip Planning Form

6.2.4 Dashboard (My Trips)

The dashboard provides personalized trip management features including:

- Saved itineraries
- Travel history
- Soft-delete functionality
- Trip restoration

The soft-delete mechanism prevents accidental data loss by moving deleted trips into a recyclable trash system instead of permanently removing them.

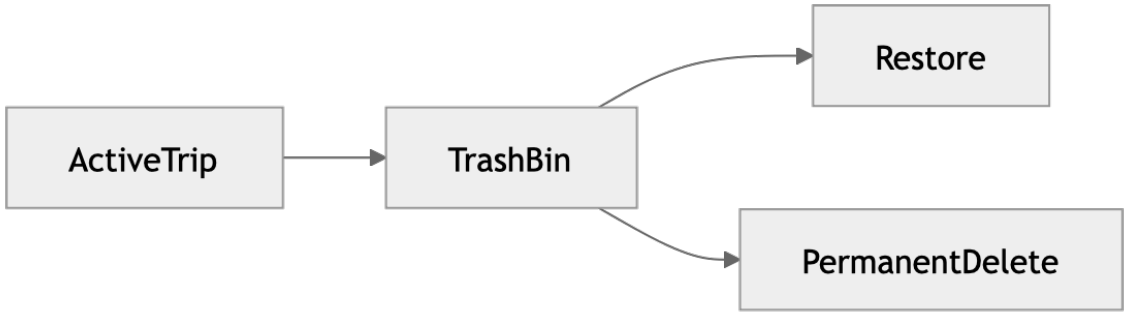


Figure 6.3: Soft Delete Workflow

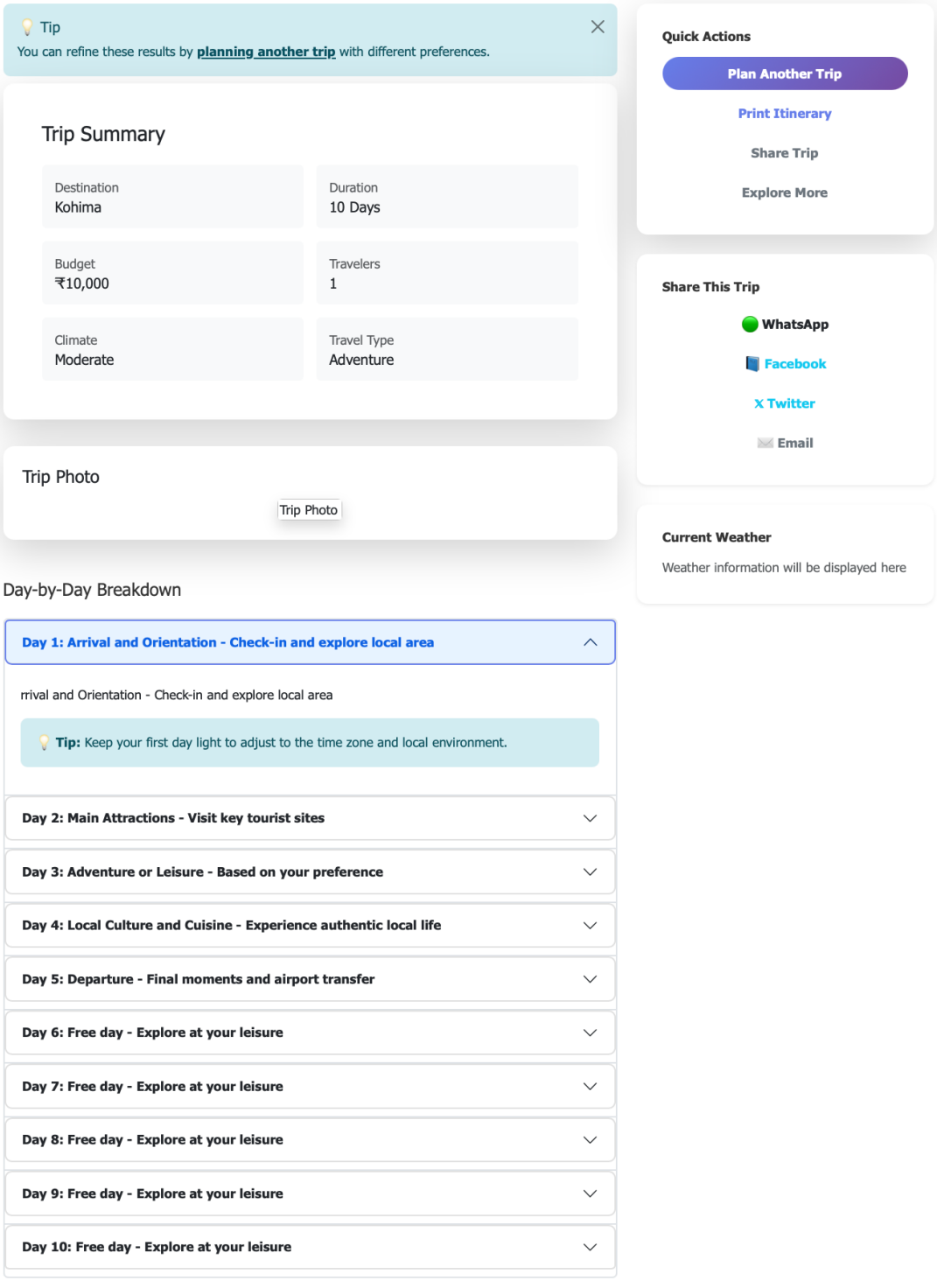
6.2.5 Results View

The results interface displays:

- Predicted destination
- Budget summaries
- Daily itinerary schedules
- Activity suggestions
- Travel guidance

The application visualizes budget distribution using percentages and organized layout structures to improve readability and user understanding.

The interface is designed to create an immersive and professional travel planning experience while maintaining performance and responsiveness across all supported devices.



My Saved Trips

Review and revisit past trip plans.

Plan a New Trip

Photo	Destination	Created	Budget	Days	Type		
	Kohima	May 17, 2026 16:49	₹10,000	10	Adventure	View	Delete
	Kohima	May 17, 2026 16:49	₹10,000	10	Adventure	View	Delete
	Kohima	May 17, 2026 16:49	₹10,000	10	Adventure	View	Delete
	Kohima	May 17, 2026 16:49	₹10,000	10	Adventure	View	Delete
	Kohima	May 17, 2026 16:49	₹10,000	10	Adventure	View	Delete
	Kohima	May 17, 2026 16:49	₹10,000	10	Adventure	View	Delete
	Goa	Feb 24, 2026 12:36	₹20,000	1	Relaxation	View	Delete
	Goa	Feb 23, 2026 16:16	₹20,000	1	Relaxation	View	Delete

Chapter 7: Testing and Evaluation

Testing and evaluation are essential phases in the software development lifecycle because they ensure that the application functions correctly, securely, and efficiently under different conditions. The AI-Based Trip Planner Web Application undergoes multiple levels of testing to validate its machine learning predictions, backend functionality, database interactions, API responses, frontend rendering, and overall user experience.

This chapter discusses the methodologies used to test the system, including unit testing, integration testing, and performance evaluation. The testing process ensures that all components of the application operate reliably and meet the project objectives.

7.1 Unit Testing

Unit testing focuses on validating individual modules and functions independently to ensure they behave as expected. In the AI-Based Trip Planner Web Application, unit tests are implemented to verify the correctness of critical backend logic, machine learning utilities, itinerary generation mechanisms, and data transformation functions.

The testing process is primarily conducted using Python's built-in unittest framework and Django's testing utilities.

Objectives of Unit Testing

The major objectives of unit testing include:

- Detecting logical errors in isolated components
- Validating data transformation functions
- Ensuring accurate cost calculations
- Verifying machine learning preprocessing
- Maintaining code reliability during updates

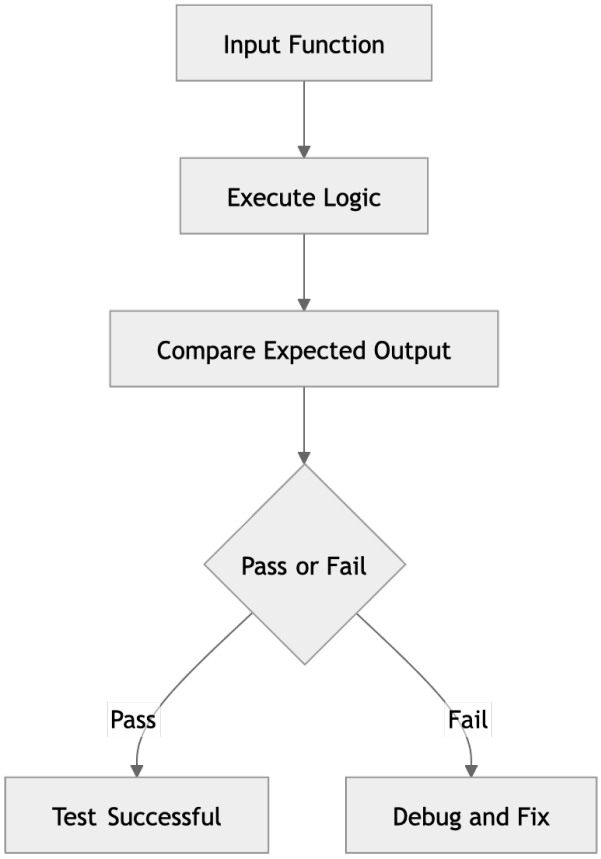


Figure 7.1: Unit Testing Workflow

7.1.1 Testing the Dynamic Itinerary Generator

The generate_trip_plan module is one of the most critical components of the application. Unit tests are designed to validate:

- Budget calculations
- Cost multipliers
- Itinerary structures
- Day allocation logic
- Percentage distributions

One important validation ensures that the generated budget breakdown percentages always total exactly 100%.

Example:

Accommodation = 40%
Food = 25%
Transportation = 20%
Activities = 15%

Total = 100%

This test prevents logical inconsistencies in financial reporting and ensures accurate travel budgeting.

7.1.2 Testing LabelEncoder Transformations

The machine learning pipeline depends heavily on proper categorical encoding. Unit tests verify that all categorical values are transformed correctly using the LabelEncoder mechanism provided by Scikit-learn.

Examples include validating:

Original Value	Encoded Value
Hot	0
Cold	1
Moderate	2

Incorrect encoding could result in inaccurate destination predictions; therefore, these tests are essential for maintaining recommendation quality.

7.1.3 Testing Database Models

The Django ORM models are also tested to ensure proper field creation and database interactions.

Model tests verify:

- Trip object creation
- Foreign key relationships
- JSONField storage
- Soft-delete functionality
- Timestamp generation

Example validation:

```
trip = Trip.objects.create(
   destination="Manali",
   budget=25000
)
```

The test confirms that valid trip records are correctly stored in the SQLite database.

7.2 Integration Testing

While unit testing validates isolated components, integration testing ensures that different modules interact correctly within the complete system architecture. The AI-Based Trip Planner Web Application integrates multiple technologies including Django, machine learning models, serializers, APIs, and SQLite databases. Therefore, testing communication between these modules is critical.

Integration tests simulate real application workflows from frontend requests to backend responses and database storage.

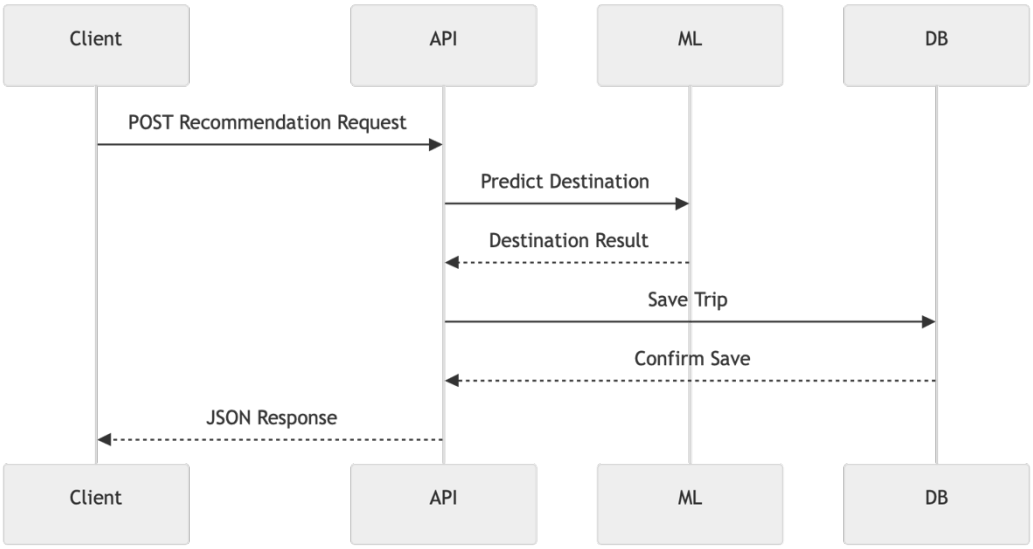


Figure 7.2: Integration Testing Architecture

7.2.1 API Endpoint Testing

The recommendation endpoint is one of the most important integration points in the application.

Endpoint:

/api/recommendation/

Integration tests verify that:

- Requests are accepted correctly
- User data is validated
- Machine learning predictions execute successfully
- Itineraries are generated properly
- Trip records are saved in the database
- Correct JSON responses are returned

7.2.2 Database Interaction Testing

Integration testing also verifies communication between Django ORM models and the SQLite database.

These tests ensure that:

- Trip records are stored correctly
- JSON itinerary data is serialized properly
- User-specific queries function correctly
- Soft-delete updates are applied successfully

Example validation:

Trip object successfully created
Trip linked to authenticated user
Itinerary JSON stored correctly

7.2.3 Authentication and Session Testing

The authentication system is tested to validate:

- User registration
- Login/logout functionality
- Session persistence
- Access restrictions
- Route protection using @login_required

These tests ensure that only authenticated users can access sensitive travel planning features.

7.3 Model Accuracy and Performance

The machine learning recommendation engine is evaluated based on prediction accuracy, inference speed, reliability, and overall system responsiveness.

The application uses a Decision Tree classification model trained on synthetically generated travel datasets. Due to the structured and well-defined nature of the dataset, the model achieves very high predictive performance.

7.3.1 Accuracy Evaluation

The dataset is divided into training and testing subsets using standard machine learning evaluation practices.

Training ratio example:

80% Training Data
20% Testing Data

The Decision Tree classifier achieves accuracy levels exceeding 95% on testing datasets.

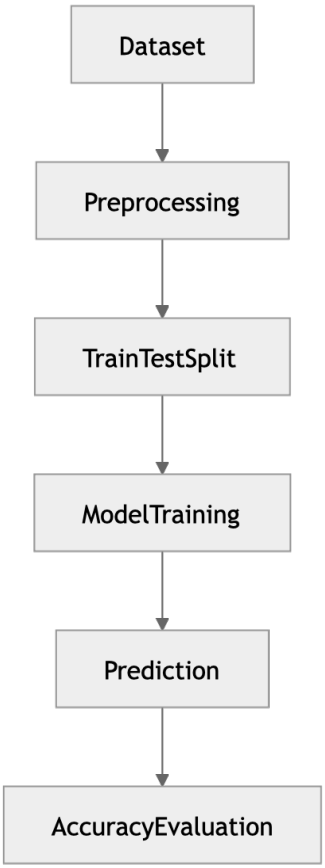


Figure 7.3: Model Training and Evaluation Pipeline

Reasons for High Accuracy

The model performs exceptionally well because:

- The synthetic dataset follows structured patterns
- Destination categories are clearly separated
- Encoded features reduce ambiguity
- Decision Trees handle categorical data efficiently

The model successfully maps:

- Budget ranges
- Climate preferences
- Travel categories
- Trip durations

to appropriate destination predictions.

7.3.2 Inference Speed and Runtime Performance

One of the most important aspects of user experience is application responsiveness. The AI-Based Trip Planner achieves extremely fast inference performance due to:

- Lightweight Decision Tree architecture
- Preloaded in-memory serialized models
- Efficient Django backend processing

Average prediction time:

< 10 milliseconds

This allows users to receive recommendations almost instantly after submitting their travel preferences.

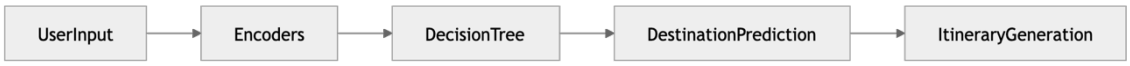


Figure 7.4: Prediction Runtime Workflow

7.3.3 System Scalability Considerations

Although the current prototype handles 36 destinations efficiently, scalability analysis indicates that the architecture can support larger datasets with additional optimization.

Potential scalability improvements include:

- Database indexing
- Model caching
- Asynchronous processing
- Cloud deployment

- Load balancing

The modular architecture ensures that future enhancements can be integrated without major structural redesign.

Chapter 8: Conclusion and Future Scope

This chapter summarizes the overall achievements of the AI-Based Trip Planner Web Application and discusses possible future enhancements that could improve scalability, intelligence, and commercial applicability.

The project successfully demonstrates how machine learning, intelligent recommendation systems, and modern web development frameworks can be combined to automate and personalize travel planning.

8.1 Summary of Contributions

The AI-Based Trip Planner Web Application successfully achieves its primary objectives by integrating machine learning-based recommendation systems with a modern web application architecture. The project demonstrates the practical viability of combining artificial intelligence with tourism services to simplify travel planning and improve user experience.

The major contributions of the project include:

1. Intelligent Destination Recommendation System

The application implements a machine learning-based recommendation engine capable of predicting travel destinations according to:

- Budget constraints
- Climate preferences
- Trip duration
- Travel categories

The use of the Scikit-learn Decision Tree classifier provides fast and accurate predictions suitable for real-time web applications.

2. Dynamic Itinerary Generation

The project introduces a rule-based itinerary generation system capable of automatically creating:

- Day-wise travel schedules
- Budget breakdowns
- Activity recommendations
- Tourist attraction plans

This reduces the manual effort traditionally required in travel planning.

3. Full-Stack Web Application Development

The system demonstrates successful integration of:

- Backend development using Django
- Frontend rendering using HTML, CSS, and JavaScript
- SQLite database management
- REST API communication
- Machine learning inference

The modular architecture ensures maintainability and scalability.

4. User-Centric Features

The application includes several modern usability features such as:

- User authentication
- Session management
- Saved trip dashboards
- Soft-delete mechanisms
- Responsive interfaces

These features improve reliability and enhance user experience.

5. Real-World Applicability

The project proves that machine learning classifiers can be effectively integrated into standard web architectures to solve real-world problems in tourism and recommendation systems.

The platform abstracts the complexity of travel planning by generating personalized recommendations and mathematically calculated budget estimations instantly.

8.2 Future Enhancements

Although the current implementation successfully demonstrates the core functionality of intelligent travel planning, several improvements can further enhance the system's capabilities and commercial readiness.

8.2.1 Global Dataset Expansion

Currently, the system supports 36 curated Indian destinations. Future development can expand the recommendation engine to include:

- International tourism destinations
- Real-world travel datasets
- Seasonal tourism trends
- Dynamic tourism metadata

Larger datasets may require upgrading from Decision Trees to more advanced ensemble algorithms such as:

- Random Forest
- XGBoost
- Gradient Boosting

These models would improve scalability and predictive accuracy for large-scale deployment.

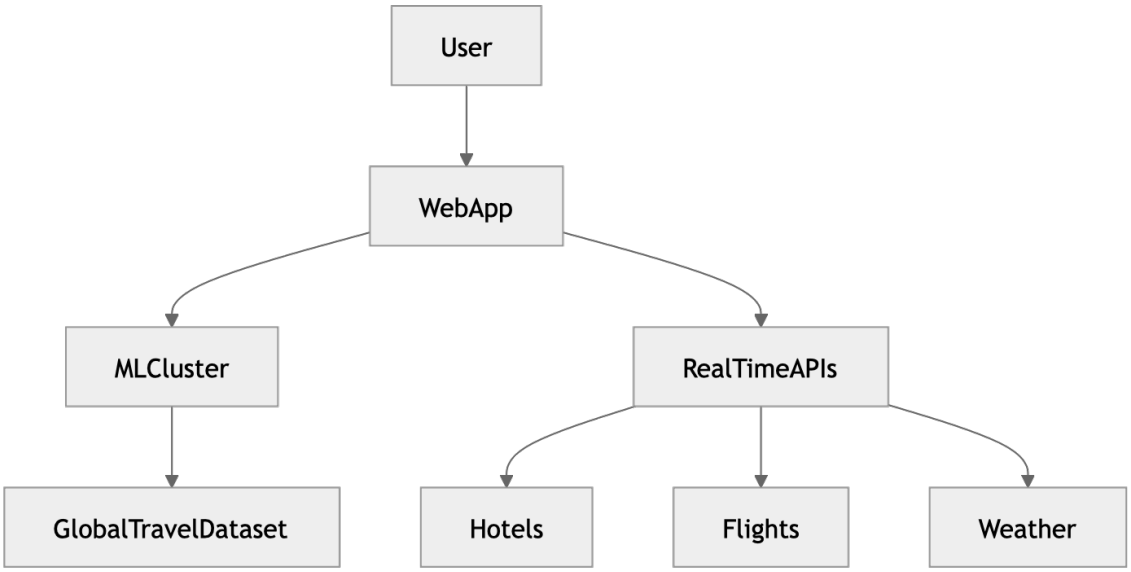


Figure 8.1: Future Scalable Architecture

8.2.2 Real-Time Price API Integration

The current system uses static cost estimation rules. Future versions could integrate live pricing APIs from travel service providers.

Possible integrations include:

- Amadeus
- Skyscanner

This would enable:

- Live flight prices
- Hotel availability
- Real-time travel cost estimation
- Dynamic package generation

8.2.3 Collaborative Filtering Recommendations

Future implementations may integrate collaborative filtering techniques similar to those used by:

- Netflix
- Amazon

Such systems could recommend destinations based on:

- Similar user behavior
- Ratings and reviews
- Historical travel trends
- Community preferences

This would improve recommendation personalization significantly.

8.2.4 Export and Sharing Features

Additional user convenience features could include:

- PDF itinerary generation
- Social media sharing
- Google Calendar synchronization
- Email itinerary delivery

Integration with APIs such as:

- Google Calendar API

would allow travelers to manage plans directly from their calendars and mobile devices.

8.2.5 Progressive Web Application (PWA) Support

Future frontend improvements may convert the application into a Progressive Web App (PWA).

Benefits include:

- Offline itinerary access
- Mobile installation support
- Faster loading speeds
- Push notifications
- Better mobile experiences

Service Workers and local caching mechanisms would allow travelers to access saved itineraries even in low-network environments.

REFERENCES

1. Django Documentation- Django Software Foundation. (2026).
2. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.
3. McKinney, W. (2010). *Data Structures for Statistical Computing in Python*. Proceedings of the 9th Python in Science Conference, 51–56.
4. Ricci, F., Rokach, L., & Shapira, B. (2015). *Recommender Systems Handbook*. Springer.
5. Buhalis, D., & Law, R. (2008). *Progress in information technology and tourism management: 20 years on and 10 years after the Internet—The state of eTourism research*. Tourism Management, 29(4), 609–623.

APPENDIX A: SOURCE CODE

Train.model.py

```
import argparse
import random
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
import joblib
import os

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_PATH = os.path.join(BASE_DIR, "data", "trip_dataset.csv")

# ----------------------------
# 1. DESTINATION RULES (used only for synthetic fallback)
# All 36 Indian States and Union Territories with representative tourist destinations
# ----------------------------
DESTINATIONS = [
    ("Tirupati", "hot", "cultural", 15000, 35000),
    ("Tawang", "cold", "adventure", 25000, 50000),
    ("Kaziranga", "hot", "adventure", 20000, 45000),
    ("Bodh Gaya", "hot", "religious", 12000, 30000),
    ("Chitrakote", "hot", "relaxation", 10000, 25000),
    ("Baga Beach", "hot", "relaxation", 15000, 40000),
    ("Gir Wildlife Sanctuary", "hot", "adventure", 18000, 40000),
    ("Kurukshetra", "hot", "cultural", 10000, 25000),
    ("Shimla", "cold", "family", 15000, 35000),
    ("Ranchi", "moderate", "family", 12000, 30000),
    ("Hampi", "moderate", "cultural", 12000, 30000),
    ("Alleppey Backwaters", "hot", "relaxation", 20000, 45000),
    ("Khajuraho", "hot", "cultural", 15000, 35000),
    ("Ajanta Caves", "hot", "cultural", 12000, 30000),
    ("Loktak Lake", "moderate", "relaxation", 12000, 28000),
    ("Cherrapunji", "moderate", "adventure", 15000, 35000),
    ("Aizawl", "moderate", "cultural", 12000, 28000),
    ("Kohima", "moderate", "cultural", 12000, 28000),
    ("Puri", "hot", "religious", 12000, 30000),
    ("Golden Temple", "hot", "religious", 10000, 25000),
    ("Jaipur", "hot", "cultural", 12000, 35000),
    ("Gangtok", "moderate", "family", 15000, 35000),
    ("Mahabalipuram", "hot", "cultural", 12000, 30000),
    ("Hyderabad", "hot", "cultural", 12000, 30000),
    ("Ujjayanta Palace", "hot", "cultural", 10000, 25000),
    ("Agra", "hot", "cultural", 12000, 35000),
    ("Rishikesh", "moderate", "adventure", 10000, 28000),
    ("Darjeeling", "cold", "family", 15000, 35000),
    ("Port Blair", "hot", "relaxation", 20000, 45000),
    ("Rock Garden", "moderate", "family", 10000, 25000),
    ("Daman Beach", "hot", "relaxation", 12000, 30000),
```

```
    ("Srinagar", "cold", "relaxation", 20000, 45000),
    ("Pangong Lake", "cold", "adventure", 30000, 60000),
    ("Minicoy Island", "hot", "relaxation", 25000, 50000),
    ("Pondicherry", "hot", "relaxation", 12000, 30000),
    ("India Gate", "hot", "cultural", 10000, 30000),
]

# ----------------------------
# 2. DATASET LOADING / GENERATION
# ----------------------------

if os.path.exists(DATA_PATH):
    # load a dataset exported from Kaggle (or any other source) into the training folder
    data = pd.read_csv(DATA_PATH)
    # basic cleaning: ensure important columns are present and numeric where expected
    expected_cols = {"budget", "days", "climate", "travel_type", "destination"}
    if not expected_cols.issubset(set(data.columns)):
        raise ValueError(f"Dataset at {DATA_PATH} missing required columns: {expected_cols - set(data.columns)}")
    # coerce numeric fields and drop rows with missing values
    data["budget"] = pd.to_numeric(data["budget"], errors="coerce")
    data["days"] = pd.to_numeric(data["days"], errors="coerce")
    data = data.dropna(subset=["budget", "days", "climate", "travel_type", "destination"])

    # remove image_url column if it exists
    if "image_url" in data.columns:
        data = data.drop(columns=["image_url"])

    print(f"Loaded dataset from {DATA_PATH}:", data.shape)
else:

    rows = []

    for _ in range(5000):
        destination, climate, travel_type, min_budget, max_budget = random.choice(DESTINATIONS)

        budget = random.randint(min_budget, max_budget)
        days = random.randint(2, 7)

        rows.append([
            budget,
            days,
            climate,
            travel_type,
            destination
        ])

    data = pd.DataFrame(
        rows,
        columns=["budget", "days", "climate", "travel_type", "destination"]
    )
```

```
    print("Synthetic dataset generated:", data.shape)

# ----------------------------
# 3. ENCODING
# ----------------------------
le_climate = LabelEncoder()
le_travel = LabelEncoder()
le_destination = LabelEncoder()

data["climate"] = le_climate.fit_transform(data["climate"])
data["travel_type"] = le_travel.fit_transform(data["travel_type"])
data["destination"] = le_destination.fit_transform(data["destination"])

X = data[["budget", "days", "climate", "travel_type"]]
y = data["destination"]

# ----------------------------
# 4. TRAIN MODEL
# ----------------------------
model = DecisionTreeClassifier(
    max_depth=8,
    random_state=42
)

model.fit(X, y)

# ----------------------------
# 5. SAVE MODEL FILES
# ----------------------------
joblib.dump(model, os.path.join(BASE_DIR, "trip_model.pkl"))
joblib.dump(le_climate, os.path.join(BASE_DIR, "climate_encoder.pkl"))
joblib.dump(le_travel, os.path.join(BASE_DIR, "travel_encoder.pkl"))
joblib.dump(le_destination, os.path.join(BASE_DIR, "destination_encoder.pkl"))

print(f"Model training complete. Model saved for {le_destination.classes_.size} destinations.")
```

service.py

```
import os
import joblib
import numpy as np
from django.conf import settings
```

```
# Load model paths
ML_FOLDER = os.path.join(settings.BASE_DIR, "trips", "ml")

model = joblib.load(os.path.join(ML_FOLDER, "trip_model.pkl"))
climate_encoder = joblib.load(os.path.join(ML_FOLDER, "climate_encoder.pkl"))
travel_encoder = joblib.load(os.path.join(ML_FOLDER, "travel_encoder.pkl"))
destination_encoder = joblib.load(os.path.join(ML_FOLDER, "destination_encoder.pkl"))


def predict_destination(budget, days, climate, travel_type):
    # Encode categorical values
    climate_encoded = climate_encoder.transform([climate])[0]
    travel_encoded = travel_encoder.transform([travel_type])[0]

    # Prepare input
    input_data = np.array([[budget, days, climate_encoded, travel_encoded]])

    # Predict
    prediction = model.predict(input_data)

    # Decode result
    destination = destination_encoder.inverse_transform(prediction)[0]

    return destination
```

urls.py

```
from django.urls import path
from .views import (
    TripRecommendationView,
    home_page,
    destinations_page,
    plan_trip_page,
    trip_results_page,
    my_trips_page,
    trip_detail_page,
    delete_trip,
    recycle_trip,
    login_page,
    logout_page,
    register_page,
)

urlpatterns = [
    path('', home_page, name='home'),
    path('login/', login_page, name='login'),
    path('logout/', logout_page, name='logout'),
    path('register/', register_page, name='register'),
    path('destinations/', destinations_page, name='destinations'),
    path('plan-trip/', plan_trip_page, name='plan-trip'),
    path('results/', trip_results_page, name='trip-results'),
```

```
    path('my-trips/', my_trips_page, name='my-trips'),
    path('trips/<int:trip_id>/', trip_detail_page, name='trip-detail'),
    path('trips/<int:trip_id>/delete/', delete_trip, name='delete-trip'),
    path('trips/<int:trip_id>/recycle/', recycle_trip, name='recycle-trip'),
    path('recommend/', TripRecommendationView.as_view(), name='recommend-trip'),
]
```

App.py

```
from django.apps import AppConfig


class TripsConfig(AppConfig):
    name = 'trips'
```

serializers.py

```
from rest  framework import serializers
from .models import Trip


class TripSerializer(serializers.ModelSerializer):
    class Meta:
        model = Trip
        fields = "__all__"
        read_only_fields = ["itinerary", "created_at", "plan_data"]


class TripRequestSerializer(serializers.Serializer):
    days = serializers.IntegerField()
    budget = serializers.FloatField()
    climate = serializers.CharField()
    travel_type = serializers.CharField()
    travelers = serializers.IntegerField(required=False, default=1)
    accommodation = serializers.CharField(required=False)
    interests = serializers.ListField(child=serializers.CharField(), required=False, default=list)
    special_requests = serializers.CharField(required=False, allow_blank=True)
    image = serializers.ImageField(required=False, allow_null=True)
```

models.py

```
from django.db import models
from django.core.files.storage import default_storage
from django.utils import timezone


class Trip(models.Model):
    CLIMATE_CHOICES = [
        ('hot', 'Hot'),
        ('cold', 'Cold'),
```

```
        ('moderate', 'Moderate'),
    ]

    TRAVEL_TYPE_CHOICES = [
        ('adventure', 'Adventure'),
        ('relaxation', 'Relaxation'),
        ('cultural', 'Cultural'),
        ('family', 'Family'),
    ]

    destination = models.CharField(max_length=255, default='Unknown')
    budget = models.IntegerField(default=0)
    days = models.IntegerField(default=1)
    climate = models.CharField(max_length=20, choices=CLIMATE_CHOICES, default='moderate', null=True,
blank=True)
    travel_type = models.CharField(max_length=20, choices=TRAVEL_TYPE_CHOICES, default='relaxation',
null=True, blank=True)
    itinerary = models.TextField(blank=True)

    # Full structured trip plan (used to render the results page and export)
    plan_data = models.JSONField(default=dict, blank=True)

    # Trip image (optional)
    image = models.ImageField(upload_to='trip_images/', blank=True, null=True)

    # Soft delete field
    is_deleted = models.BooleanField(default=False)
    deleted_at = models.DateTimeField(null=True, blank=True)

    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.destination} - {self.budget}"

    def delete(self, *args, **kwargs):
        # Soft delete: mark as deleted instead of actually deleting
        self.is_deleted = True
        self.deleted_at = timezone.now()
        self.save()

    def recycle(self):
        # Restore a soft-deleted trip
        self.is_deleted = False
        self.deleted_at = None
        self.save()

    def hard_delete(self, *args, **kwargs):
        # Actually delete the trip and its image file
        if self.image:
            default_storage.delete(self.image.name)
        super().delete(*args, **kwargs)
```

views.py

```
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from rest_framework.decorators import api_view
from django.views.decorators.csrf import csrf_exempt
from django.utils.decorators import method_decorator
from .serializers import TripRequestSerializer
from .services.ai_engine import predict_destination, generate_trip_plan
from .models import Trip
import json
import os
from urllib.parse import unquote

from django.conf import settings
from django.shortcuts import render, redirect, get_object_or_404
from django.urls import reverse
from django.contrib import messages
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth.models import User
from django.contrib.auth.decorators import login_required

def get_destination_image_url(destination, default_query):
    """Return a local static image if available, otherwise fallback to Unsplash."""
    filename = f"{destination.lower().replace(' ', '_')}.jpg"
    local_path = os.path.join(settings.BASE_DIR, "static", "images", filename)
    if os.path.exists(local_path):
        static_url = settings.STATIC_URL
        if not static_url.startswith('/'):
            static_url = '/' + static_url
        static_url = static_url.rstrip('/') + '/'
        return f"{static_url}images/{filename}"
    return f"https://source.unsplash.com/600x400/?{default_query}"


def login_page(request):
    """Handle user login"""
    if request.method == 'POST':
        username = request.POST.get('username')
        password = request.POST.get('password')
        user = authenticate(request, username=username, password=password)
        if user is not None:
            login(request, user)
            messages.success(request, f'Welcome back, {username}!')
            return redirect('home')
        else:
            messages.error(request, 'Invalid username or password.')
    return render(request, 'login.html')
```

```
def register_page(request):
    """Handle user registration"""
    if request.method == 'POST':
        username = request.POST.get('username')
        email = request.POST.get('email')
        password = request.POST.get('password')
        password_confirm = request.POST.get('password_confirm')

        if password != password_confirm:
            messages.error(request, 'Passwords do not match.')
            return render(request, 'register.html')

        if User.objects.filter(username=username).exists():
            messages.error(request, 'Username already exists.')
            return render(request, 'register.html')

        if User.objects.filter(email=email).exists():
            messages.error(request, 'Email already exists.')
            return render(request, 'register.html')

        user = User.objects.create_user(username=username, email=email, password=password)
        login(request, user)
        messages.success(request, f'Welcome, {username}! Your account has been created.')
        return redirect('home')

    return render(request, 'register.html')

def logout_page(request):
    """Handle user logout"""
    logout(request)
    messages.success(request, 'You have been logged out successfully.')
    return redirect('login')

def home_page(request):
    """Show landing page for anonymous users, home page for authenticated users"""
    if not request.user.is_authenticated:
        return render(request, "landing.html")
    return render(request, "home.html")

@login_required(login_url='login')
def destinations_page(request):
    """Display all available destinations from trained model"""
    destinations_data = [
        {'name': 'Tirupati', 'image_file': 'tirupati.jpg', 'state': 'Andhra Pradesh', 'climate': 'hot', 'best_time': 'Sep - Mar',
'description': 'Ancient temple with spiritual significance'},
        {'name': 'Tawang', 'image_file': 'tawang.jpg', 'state': 'Arunachal Pradesh', 'climate': 'cold', 'best_time': 'Apr - Jun',
'description': 'Scenic monastery and mountain beauty'},
        {'name': 'Kaziranga', 'image_file': 'kaziranga.jpg', 'state': 'Assam', 'climate': 'hot', 'best_time': 'Nov - Apr',
'description': 'Wildlife sanctuary with one-horned rhinos'},
```

```
    {'name': 'Bodh Gaya', 'image_file': 'bodh_gaya.jpg', 'state': 'Bihar', 'climate': 'hot', 'best_time': 'Oct - Mar', 'description': 'Pilgrimage destination and meditation site'},
    {'name': 'Chitrakote', 'image_file': 'chitrakote.jpg', 'state': 'Chhattisgarh', 'climate': 'hot', 'best_time': 'Oct - Mar', 'description': 'Stunning waterfall and natural beauty'},
    {'name': 'Baga Beach', 'image_file': 'baga_beach.jpg', 'state': 'Goa', 'climate': 'hot', 'best_time': 'Nov - Feb', 'description': 'Vibrant beach with nightlife and water sports'},
    {'name': 'Gir Wildlife Sanctuary', 'image_file': 'gir_wildlife_sanctuary.jpg', 'state': 'Gujarat', 'climate': 'hot', 'best_time': 'Dec - Apr', 'description': 'Home to Asiatic lions and wildlife'},
    {'name': 'Kurukshetra', 'image_file': 'kurukshetra.jpg', 'state': 'Haryana', 'climate': 'hot', 'best_time': 'Oct - Mar', 'description': 'Historic pilgrimage and cultural site'},
    {'name': 'Shimla', 'image_file': 'shimla.jpg', 'state': 'Himachal Pradesh', 'climate': 'cold', 'best_time': 'Mar - Jun', 'description': 'Hill station with colonial charm'},
    {'name': 'Ranchi', 'image_file': 'ranchi.jpg', 'state': 'Jharkhand', 'climate': 'moderate', 'best_time': 'Oct - Mar', 'description': 'Waterfalls and natural parks'},
    {'name': 'Hampi', 'image_file': 'hampi.jpg', 'state': 'Karnataka', 'climate': 'moderate', 'best_time': 'Oct - Feb', 'description': 'UNESCO heritage ruins and architecture'},
    {'name': 'Alleppey Backwaters', 'image_file': 'alleppey_backwaters.jpg', 'state': 'Kerala', 'climate': 'hot', 'best_time': 'Sep - Mar', 'description': 'Houseboat cruises and backwater beauty'},
    {'name': 'Khajuraho', 'image_file': 'khajuraho.jpg', 'state': 'Madhya Pradesh', 'climate': 'hot', 'best_time': 'Oct - Mar', 'description': 'Ancient temples with intricate carvings'},
    {'name': 'Ajanta Caves', 'image_file': 'ajanta_caves.jpg', 'state': 'Maharashtra', 'climate': 'hot', 'best_time': 'Nov - Mar', 'description': 'Ancient Buddhist cave complex'},
    {'name': 'Loktak Lake', 'image_file': 'loktak_lake.jpg', 'state': 'Manipur', 'climate': 'moderate', 'best_time': 'Oct - Mar', 'description': 'Unique floating lake with natural beauty'},
    {'name': 'Cherrapunji', 'image_file': 'cherrapunji.jpg', 'state': 'Meghalaya', 'climate': 'moderate', 'best_time': 'Oct - May', 'description': 'Waterfalls and green landscapes'},
    {'name': 'Aizawl', 'image_file': 'aizawl.jpg', 'state': 'Mizoram', 'climate': 'moderate', 'best_time': 'Oct - May', 'description': 'Hill city with cultural richness'},
    {'name': 'Kohima', 'image_file': 'kohima.jpg', 'state': 'Nagaland', 'climate': 'moderate', 'best_time': 'Oct - Apr', 'description': 'Cultural hub and festival destination'},
    {'name': 'Puri', 'image_file': 'puri.jpg', 'state': 'Odisha', 'climate': 'hot', 'best_time': 'Oct - Mar', 'description': 'Sacred temple and beach town'},
    {'name': 'Golden Temple', 'image_file': 'golden_temple.jpg', 'state': 'Punjab', 'climate': 'hot', 'best_time': 'Oct - Mar', 'description': 'Holy pilgrimage site and spiritual center'},
    {'name': 'Jaipur', 'image_file': 'jaipur.jpg', 'state': 'Rajasthan', 'climate': 'hot', 'best_time': 'Oct - Mar', 'description': 'City palace and historic forts'},
    {'name': 'Gangtok', 'image_file': 'gangtok.jpg', 'state': 'Sikkim', 'climate': 'moderate', 'best_time': 'Mar - Jun', 'description': 'Mountain city with Buddhist monasteries'},
    {'name': 'Mahabalipuram', 'image_file': 'mahabalipuram.jpg', 'state': 'Tamil Nadu', 'climate': 'hot', 'best_time': 'Nov - Feb', 'description': 'Heritage temples and beach town'},
    {'name': 'Hyderabad', 'image_file': 'hyderabad.jpg', 'state': 'Telangana', 'climate': 'hot', 'best_time': 'Oct - Mar', 'description': 'Historic monuments and tech city'},
    {'name': 'Ujjayanta Palace', 'image_file': 'ujjayanta_palace.jpg', 'state': 'Tripura', 'climate': 'hot', 'best_time': 'Oct - Mar', 'description': 'Historic palace and museum'},
    {'name': 'Agra', 'image_file': 'agra.jpg', 'state': 'Uttar Pradesh', 'climate': 'hot', 'best_time': 'Oct - Mar', 'description': 'Taj Mahal and iconic monument'},
    {'name': 'Rishikesh', 'image_file': 'rishikesh.jpg', 'state': 'Uttarakhand', 'climate': 'moderate', 'best_time': 'Sep - Nov', 'description': 'Yoga capital and adventure hub'},
    {'name': 'Darjeeling', 'image_file': 'darjeeling.jpg', 'state': 'West Bengal', 'climate': 'cold', 'best_time': 'Mar - Jun', 'description': 'Tea gardens and mountain railway'},
```

```
        {'name': 'Port Blair', 'image_file': 'port_blair.jpg', 'state': 'Andaman and Nicobar Islands', 'climate': 'hot',
'best_time': 'Oct - May', 'description': 'Island paradise and beaches'},
        {'name': 'Rock Garden', 'image_file': 'rock_garden.jpg', 'state': 'Chandigarh', 'climate': 'moderate', 'best_time': 'Sep
- Mar', 'description': 'Artistic rock garden creation'},
        {'name': 'Daman Beach', 'image_file': 'daman_beach.jpg', 'state': 'Dadra and Nagar Haveli and Daman and Diu',
'climate': 'hot', 'best_time': 'Oct - Mar', 'description': 'Beach resort and watersports'},
        {'name': 'Srinagar', 'image_file': 'srinagar.jpg', 'state': 'Jammu and Kashmir', 'climate': 'cold', 'best_time': 'Apr -
Oct', 'description': 'Houseboats and scenic lakes'},
        {'name': 'Pangong Lake', 'image_file': 'pangong_lake.jpg', 'state': 'Ladakh', 'climate': 'cold', 'best_time': 'Jun - Sep',
'description': 'High altitude lake and trekking'},
        {'name': 'Minicoy Island', 'image_file': 'minicoy_island.jpg', 'state': 'Lakshadweep', 'climate': 'hot', 'best_time': 'Oct
- Mar', 'description': 'Tropical island paradise'},
        {'name': 'Pondicherry', 'image_file': 'pondicherry.jpg', 'state': 'Puducherry', 'climate': 'hot', 'best_time': 'Oct - Mar',
'description': 'French colonial charm and beaches'},
        {'name': 'India Gate', 'image_file': 'india_gate.jpg', 'state': 'Delhi', 'climate': 'hot', 'best_time': 'Oct - Mar',
'description': 'Iconic monument and city landmark'},
    ]
    # Add a cache-busting version (file mtime) for each image so browser picks up updates
    for d in destinations_data:
        filename = d.get('image_file')
        if filename:
            local_path = os.path.join(settings.BASE_DIR, 'static', 'images', filename)
            try:
                d['image_version'] = str(int(os.path.getmtime(local_path)))
            except Exception:
                d['image_version'] = '0'
        else:
            d['image_version'] = '0'

    return render(request, "destinations.html", {'destinations': destinations_data})

@login_required(login_url='login')
def plan_trip_page(request):
    """Display trip planning form"""
    return render(request, "plan_trip.html")


@login_required(login_url='login')
def my_trips_page(request):
    """Display a list of saved trips"""
    trips = Trip.objects.filter(is_deleted=False).order_by('-created_at')
    deleted_trips = Trip.objects.filter(is_deleted=True).order_by('-deleted_at')
    return render(request, "my_trips.html", {"trips": trips, "deleted_trips": deleted_trips})


@login_required(login_url='login')
def delete_trip(request, trip_id):
    """Soft delete a trip"""
    if request.method == 'POST':
        try:
            trip = get_object_or_404(Trip, pk=trip_id, is_deleted=False)
```

```
            trip.delete()  # This will now do soft delete
            messages.success(request, f'Trip to {trip.destination} has been moved to trash.')
        except Trip.DoesNotExist:
            messages.error(request, 'Trip not found.')
    return redirect('my-trips')


@login_required(login_url='login')
def recycle_trip(request, trip_id):
    """Restore a soft-deleted trip"""
    if request.method == 'POST':
        try:
            trip = get_object_or_404(Trip, pk=trip_id, is_deleted=True)
            trip.recycle()
            messages.success(request, f'Trip to {trip.destination} has been restored.')
        except Trip.DoesNotExist:
            messages.error(request, 'Trip not found.')
    return redirect('my-trips')


@login_required(login_url='login')
def trip_detail_page(request, trip_id):
    """Display a saved trip's details"""
    trip = get_object_or_404(Trip, pk=trip_id)
    trip_plan = trip.plan_data or {}
    # If we have older trips in the DB without full plan_data, backfill key pieces
    if not trip_plan.get('destination'):
        trip_plan['destination'] = trip.destination
    if not trip_plan.get('budget'):
        trip_plan['budget'] = trip.budget
    if not trip_plan.get('days'):
        trip_plan['days'] = trip.days
    if not trip_plan.get('climate'):
        trip_plan['climate'] = trip.climate
    if not trip_plan.get('travel_type'):
        trip_plan['travel_type'] = trip.travel_type

    if not trip_plan.get('itinerary'):
        try:
            # Try to parse as JSON array first
            parsed_itinerary = json.loads(trip.itinerary)
            if isinstance(parsed_itinerary, list):
                trip_plan['itinerary'] = parsed_itinerary
            else:
                # If it's a string, convert to single-item list
                trip_plan['itinerary'] = [trip.itinerary]
        except (json.JSONDecodeError, TypeError):
            # If JSON parsing fails, treat as plain string and convert to list
            trip_plan['itinerary'] = [trip.itinerary] if trip.itinerary else []

    # Add image URL if exists
```

```
    if trip.image:
        trip_plan['image_url'] = trip.image.url

    trip_plan["trip_id"] = trip.id

    # Keep the plan in session to support the results template
    request.session["trip_plan"] = trip_plan
    return render(request, "trip_results.html", {"trip_plan": trip_plan})


@login_required(login_url='login')
def trip_results_page(request):
    """Display trip results"""

    # Allow viewing a saved trip by id
    trip_id = request.GET.get('trip_id')
    if trip_id:
        try:
            trip = Trip.objects.get(pk=int(trip_id))
            trip_data = trip.plan_data or {}
            # Backfill missing fields for trips created before plan_data was added
            if not trip_data.get('destination'):
                trip_data['destination'] = trip.destination
            if not trip_data.get('budget'):
                trip_data['budget'] = trip.budget
            if not trip_data.get('days'):
                trip_data['days'] = trip.days
            if not trip_data.get('climate'):
                trip_data['climate'] = trip.climate
            if not trip_data.get('travel_type'):
                trip_data['travel_type'] = trip.travel_type

            if not trip_data.get('itinerary'):
                try:
                    # Try to parse as JSON array first
                    parsed_itinerary = json.loads(trip.itinerary)
                    if isinstance(parsed_itinerary, list):
                        trip_data['itinerary'] = parsed_itinerary
                    else:
                        # If it's a string, convert to single-item list
                        trip_data['itinerary'] = [trip.itinerary]
                except (json.JSONDecodeError, TypeError):
                    # If JSON parsing fails, treat as plain string and convert to list
                    trip_data['itinerary'] = [trip.itinerary] if trip.itinerary else []

            # Add image URL if exists
            if trip.image:
                trip_data['image_url'] = trip.image.url

            trip_data["trip_id"] = trip.id
            request.session["trip_plan"] = trip_data
```

```
        except (Trip.DoesNotExist, ValueError):
            pass

    # Check if trip data is passed via URL parameter
    trip_data_param = request.GET.get('data')
    if trip_data_param:
        try:
            trip_data = json.loads(unquote(trip_data_param))
            request.session["trip_plan"] = trip_data
        except (json.JSONDecodeError, TypeError):
            pass

    # Get trip data from session
    trip_data = request.session.get("trip_plan")
    if not trip_data:
        # If no trip data, redirect to plan trip page
        return redirect('plan-trip')

    return render(request, "trip_results.html", {'trip_plan': trip_data})

class TripRecommendationView(APIView):
    @method_decorator(csrf_exempt)
    def dispatch(self, *args, **kwargs):
        return super().dispatch(*args, **kwargs)

    def post(self, request):
        serializer = TripRequestSerializer(data=request.data)

        if serializer.is_valid():
            data = serializer.validated_data

            # If destination is provided, use it; otherwise predict one
            if 'destination' in request.data and request.data['destination']:
                destination = request.data['destination']
            else:
                destination = predict_destination(
                    budget=data["budget"],
                    days=data["days"],
                    climate=data["climate"],
                    travel_type=data["travel_type"]
                )

            # Parse interests if it's a JSON string
            interests = request.data.get('interests', [])
            if isinstance(interests, str):
                try:
                    interests = json.loads(interests)
                except json.JSONDecodeError:
                    interests = []

            # Convert travelers to int
```

```
        try:
            travelers = int(request.data.get('travelers', 1))
        except (ValueError, TypeError):
            travelers = 1

        try:
            # Generate comprehensive trip plan
            trip_plan = generate_trip_plan(
                destination=destination,
                days=data["days"],
                budget=data["budget"],
                climate=data["climate"],
                travel_type=data["travel_type"],
                travelers=travelers,
                accommodation=request.data.get('accommodation', 'standard'),
                interests=interests,
                special_requests=request.data.get('special_requests', '')
            )

            # Save to database (including full plan data for later review)
            trip = Trip.objects.create(
                destination=trip_plan['destination'],
                budget=trip_plan['budget'],
                days=trip_plan['days'],
                climate=trip_plan['climate'],
                travel_type=trip_plan['travel_type'],
                itinerary=json.dumps(trip_plan.get('itinerary', [])),
                plan_data=trip_plan,
                image=data.get('image'),  # Save uploaded image if provided
            )

            # Return full trip plan data
            return Response({
                **trip_plan,
                "trip_id": trip.id,
            }, status=status.HTTP_200_OK)
        except Exception as e:
            return Response({'error': str(e)}, status=status.HTTP_500_INTERNAL_SERVER_ERROR)

    return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

Manage.py

Manage.py

```
#!/usr/bin/env python
"""Django's command-line utility for administrative tasks."""
import os
import sys


def main():
```

```
    """Run administrative tasks."""
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings')
    try:
        from django.core.management import execute_from_command_line
    except ImportError as exc:
        raise ImportError(
            "Couldn't import Django. Are you sure it's installed and "
            "available on your PYTHONPATH environment variable? Did you "
            "forget to activate a virtual environment?"
        ) from exc
    execute_from_command_line(sys.argv)


if __name__ == '__main__':
    main()
```

base.html

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>AI Trip Planner</title>
    <meta name="viewport" content="width=device-width, initial-scale=1">

    <!-- Bootstrap 5 CDN -->
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
    <link rel="stylesheet" href="{% static 'css/style.css' %}">

    <!-- CSRF Token for AJAX requests -->
    <meta name="csrf-token" content="{{ csrf_token }}">
</head>
<body>
    <!-- Hidden form for CSRF token -->
    <form id="csrf-form" style="display: none;">
        {% csrf_token %}
    </form>

<!-- 🔷 Navbar -->
<nav class="navbar navbar-expand-lg navbar-dark bg-dark shadow">
    <div class="container">
        <a class="navbar-brand fw-bold" href="{% url 'home' %}">✈️ AI Trip Planner</a>

        <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
                data-bs-target="#navbarNav" aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle
navigation">
            <span class="navbar-toggler-icon"></span>
        </button>
```

```
    <div class="collapse navbar-collapse" id="navbarNav">
      <ul class="navbar-nav ms-auto">
        {% if user.is_authenticated %}
          <li class="nav-item">
            <a class="nav-link" href="{% url 'home' %}">Home</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="{% url 'destinations' %}">Destinations</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="{% url 'plan-trip' %}">Plan Trip</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="{% url 'my-trips' %}">My Trips</a>
          </li>

          <li class="nav-item dropdown">
            <a class="nav-link dropdown-toggle" href="#" id="navbarDropdown" role="button" data-bs-
toggle="dropdown" aria-expanded="false">
              👤 {{ user.username }}
            </a>
            <ul class="dropdown-menu dropdown-menu-end" aria-labelledby="navbarDropdown">
              <li><a class="dropdown-item" href="{% url 'my-trips' %}">My Trips</a></li>
              <li><hr class="dropdown-divider"></li>
              <li><a class="dropdown-item" href="{% url 'logout' %}">Logout</a></li>
            </ul>
          </li>
        {% else %}
          <li class="nav-item">
            <a class="nav-link" href="{% url 'login' %}">Login</a>
          </li>
          <li class="nav-item">
            <a class="btn btn-primary ms-2" href="{% url 'register' %}">Sign Up</a>
          </li>
        {% endif %}
      </ul>
    </div>
  </div>
</nav>

<!-- Messages -->
{% if messages %}
<div class="container mt-3">
  {% for message in messages %}
  <div class="alert alert-{{ message.tags }} alert-dismissible fade show" role="alert">
    {{ message }}
    <button type="button" class="btn-close" data-bs-dismiss="alert" aria-label="Close"></button>
  </div>
  {% endfor %}
</div>
```

```
{% endif %}

{% block content %}{% endblock %}

<!-- 🔷 Footer -->
<footer class="bg-dark text-white text-center p-4 mt-5">
  <p class="mb-0">© 2026 AI Trip Planner</p>
</footer>

<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

Destinations.html

```
{% extends 'base.html' %}
{% load static %}

{% block content %}

<!-- 🌍 Destinations Header -->
<div class="bg-light py-5">
  <div class="container">
    <h1 class="display-5 fw-bold mb-3">Explore Destinations</h1>
    <p class="lead text-muted">Discover amazing places across India</p>
  </div>
</div>

<!-- 🔍 Filter Section -->
<section class="container py-4">
  <div class="row mb-4">
    <div class="col-md-4">
      <input type="text" id="searchInput" class="form-control form-control-lg" placeholder="Search
destinations...">
    </div>
    <div class="col-md-4">
      <select id="climateFilter" class="form-select form-select-lg">
        <option value="">All Climates</option>
        <option value="hot">Hot</option>
        <option value="cold">Cold</option>
        <option value="moderate">Moderate</option>
      </select>
    </div>
    <div class="col-md-4">
      <button class="btn btn-outline-primary w-100" onclick="resetFilters()">Reset Filters</button>
    </div>
  </div>
</section>

<!-- 🎴 Destinations Grid -->
```

```
<section class="container pb-5">
  <div class="row g-4" id="destinationsContainer">
    {% for destination in destinations %}
    <div class="col-md-6 col-lg-4 destination-card" data-name="{{ destination.name|lower }}" data-climate="{{
destination.climate }}">
      <div class="card h-100 shadow-sm border-0 transition-card">
        <img src="{% static 'images/' %}{{ destination.image_file }}?v={{ destination.image_version }}"
class="card-img-top" alt="{{ destination.name }}" style="height: 250px; object-fit: cover;"
onerror="this.onerror=null; this.src='https://via.placeholder.com/400x250?text=No+Image';">
        <div class="card-body">
          <h5 class="card-title fw-bold">{{ destination.name }}</h5>
          <p class="card-text text-muted small">{{ destination.state }}</p>
          <p class="card-text text-muted">{{ destination.description }}</p>

          <div class="mb-3">
            <span class="badge bg-info">{{ destination.climate|upper }}</span>
            <span class="badge bg-warning text-dark">{{ destination.best_time }}</span>
          </div>
        </div>
        <div class="card-footer bg-white border-top-0">
          <a href="{% url 'plan-trip' %}?destination={{ destination.name }}" class="btn btn-primary w-100">Plan
Trip Here</a>
        </div>
      </div>
    </div>
    {% endfor %}
  </div>

  <!-- No Results Message -->
  <div id="noResults" class="alert alert-info text-center py-5" style="display: none;">
    <h5>No destinations found</h5>
    <p>Try adjusting your filters</p>
  </div>
</section>

<!-- 📱 Features Section -->
<section class="bg-light py-5">
  <div class="container">
    <h2 class="text-center mb-5">Why Choose Our Destinations?</h2>
    <div class="row g-4">
      <div class="col-md-4 text-center">
        <div class="mb-3">
          <i class="bi bi-star-fill" style="font-size: 2rem; color: #007bff;"></i>
        </div>
        <h5>Curated Selections</h5>
        <p>Handpicked destinations for unforgettable experiences</p>
      </div>
      <div class="col-md-4 text-center">
        <div class="mb-3">
          <i class="bi bi-person-check" style="font-size: 2rem; color: #28a745;"></i>
        </div>
```

```
            <h5>Expert Recommendations</h5>
            <p>AI-powered suggestions based on your preferences</p>
          </div>
          <div class="col-md-4 text-center">
            <div class="mb-3">
              <i class="bi bi-calendar-check" style="font-size: 2rem; color: #ffc107;"></i>
            </div>
            <h5>Best Time to Visit</h5>
            <p>Know the perfect season for each destination</p>
          </div>
        </div>
      </div>
    </section>

    <!-- 🎯 CTA Section -->
    <section class="bg-primary text-white py-5">
      <div class="container text-center">
        <h2 class="mb-3">Ready to Plan Your Trip?</h2>
        <p class="lead mb-4">Let our AI help you create the perfect itinerary</p>
        <a href="{% url 'plan-trip' %}" class="btn btn-light btn-lg">Start Planning Now</a>
      </div>
    </section>

    <style>
      .transition-card {
        transition: transform 0.3s ease, box-shadow 0.3s ease;
      }

      .transition-card:hover {
        transform: translateY(-10px);
        box-shadow: 0 15px 30px rgba(0, 0, 0, 0.2) !important;
      }

      .destination-card {
        animation: fadeIn 0.5s ease-in;
      }

      @keyframes fadeIn {
        from {
          opacity: 0;
          transform: translateY(10px);
        }
        to {
          opacity: 1;
          transform: translateY(0);
        }
      }
    </style>

    <script>
      const searchInput = document.getElementById('searchInput');
```

```
  const climateFilter = document.getElementById('climateFilter');
  const destinationCards = document.querySelectorAll('.destination-card');
  const noResults = document.getElementById('noResults');

  function filterDestinations() {
    const searchTerm = searchInput.value.toLowerCase();
    const selectedClimate = climateFilter.value;
    let visibleCount = 0;

    destinationCards.forEach(card => {
      const name = card.getAttribute('data-name');
      const climate = card.getAttribute('data-climate');

      const matchesSearch = name.includes(searchTerm);
      const matchesClimate = selectedClimate === '' || climate === selectedClimate;

      if (matchesSearch && matchesClimate) {
        card.style.display = 'block';
        visibleCount++;
      } else {
        card.style.display = 'none';
      }
    });

    noResults.style.display = visibleCount === 0 ? 'block' : 'none';
  }

  function resetFilters() {
    searchInput.value = '';
    climateFilter.value = '';
    filterDestinations();
  }

  searchInput.addEventListener('keyup', filterDestinations);
  climateFilter.addEventListener('change', filterDestinations);
</script>

{% endblock %}
```

Home.html

```
{% extends 'base.html' %}
{% load static %}

{% block content %}

<!-- Hero Section -->
<section class="hero-section text-white text-center d-flex align-items-center">
  <div class="container">
    <h1 class="display-4 fw-bold">Plan Your Perfect Trip with AI</h1>
    <p class="lead">Smart itineraries. Smart budgets. Smart travel.</p>
    <a href="#planner" class="btn btn-primary btn-lg mt-3">Start Planning</a>
```

```
  </div>
</section>

<!-- 🎠 Carousel -->
<div class="container mt-5">
  <h2 class="text-center mb-4">Popular Destinations</h2>

  <div id="destinationCarousel" class="carousel slide" data-bs-ride="carousel">
    <div class="carousel-inner rounded shadow">

      <div class="carousel-item active">
        <img src="{% static 'css/images/Goa.jpeg' %}" class="d-block w-100" alt="Goa">
        <div class="carousel-caption">
          <h3>Goa</h3>
        </div>
      </div>

      <div class="carousel-item">
        <img src="{% static 'css/images/Manali.jpeg' %}" class="d-block w-100" alt="Manali">
        <div class="carousel-caption">
          <h3>Manali</h3>
        </div>
      </div>

      <div class="carousel-item">
        <img src="{% static 'css/images/Jaipur.jpg' %}" class="d-block w-100" alt="Jaipur">
        <div class="carousel-caption">
          <h3>Jaipur</h3>
        </div>
      </div>

    </div>

    <button class="carousel-control-prev" type="button" data-bs-target="#destinationCarousel" data-bs-
slide="prev">
      <span class="carousel-control-prev-icon"></span>
    </button>

    <button class="carousel-control-next" type="button" data-bs-target="#destinationCarousel" data-bs-slide="next">
      <span class="carousel-control-next-icon"></span>
    </button>
  </div>
</div>

<!-- 📝 Planner Form -->
<section id="planner" class="container mt-5">
  <div class="card shadow-lg p-4">
    <h3 class="mb-4 text-center">Generate Your Trip</h3>

    <form id="tripForm">
      <div class="row">
```

```
        <div class="col-md-6 mb-3">
          <label class="form-label">Number of Days</label>
          <input type="number" class="form-control" id="days" name="days" min="1" max="30" required>
        </div>

        <div class="col-md-6 mb-3">
          <label class="form-label">Budget (₹)</label>
          <input type="number" class="form-control" id="budget" name="budget" min="1000" required>
        </div>
      </div>

      <div class="row">
        <div class="col-md-6 mb-3">
          <label class="form-label">Climate Preference</label>
          <select class="form-select" id="climate" name="climate" required>
            <option value="">Select Climate</option>
            <option value="hot">Hot</option>
            <option value="cold">Cold</option>
            <option value="moderate">Moderate</option>
          </select>
        </div>

        <div class="col-md-6 mb-3">
          <label class="form-label">Travel Type</label>
          <select class="form-select" id="travel_type" name="travel_type" required>
            <option value="">Select Travel Type</option>
            <option value="adventure">Adventure</option>
            <option value="relaxation">Relaxation</option>
          </select>
        </div>
      </div>

      <button type="submit" class="btn btn-primary w-100">Generate Recommendation</button>
    </form>

    <div id="result" class="mt-4" style="display:none;">
      <div class="alert alert-success">
        <h5>✈️ Recommended Destination</h5>
        <h2 id="destination" class="text-primary"></h2>
        <p><strong>Budget:</strong> ₹<span id="resultBudget"></span></p>
        <p><strong>Days:</strong> <span id="resultDays"></span></p>
        <p><strong>Climate:</strong> <span id="resultClimate"></span></p>
        <p><strong>Travel Type:</strong> <span id="resultTravelType"></span></p>
      </div>
    </div>

    <script>
      document.getElementById('tripForm').addEventListener('submit', async function(e) {
        e.preventDefault();

        const formData = {
```

```
            days: parseInt(document.getElementById('days').value),
            budget: parseFloat(document.getElementById('budget').value),
            climate: document.getElementById('climate').value,
            travel_type: document.getElementById('travel_type').value
        };

        try {
            const endpoint = '{% url "recommend-trip" %}';
            const response = await fetch(endpoint, {
                method: 'POST',
                headers: {
                    'Content-Type': 'application/json',
                    'X-CSRFToken': getCookie('csrftoken')
                },
                body: JSON.stringify(formData)
            });

            const result = await response.json();

            if (response.ok) {
                const tripData = encodeURIComponent(JSON.stringify(result));
                window.location.href = `{% url "trip-results" %}?data=${tripData}`;
            } else {
                alert('Error: ' + JSON.stringify(result));
            }
        } catch (error) {
            alert('Error: ' + error.message);
        }
    });

    function getCookie(name) {
        let cookieValue = null;
        if (document.cookie && document.cookie !== '') {
            const cookies = document.cookie.split(';');
            for (let i = 0; i < cookies.length; i++) {
                const cookie = cookies[i].trim();
                if (cookie.substring(0, name.length + 1) === (name + '=')) {
                    cookieValue = decodeURIComponent(cookie.substring(name.length + 1));
                    break;
                }
            }
        }
        return cookieValue;
    }
</script>
</div>
</section>

{% endblock %}
```

Mytrips.html

```
{% extends 'base.html' %}
{% load humanize %}

{% block content %}
<div class="container py-5">
  <div class="d-flex justify-content-between align-items-center mb-4">
    <div>
      <h1 class="h3">My Saved Trips</h1>
      <p class="text-muted mb-0">Review and revisit past trip plans.</p>
    </div>
    <a href="{% url 'plan-trip' %}" class="btn btn-primary">Plan a New Trip</a>
  </div>

  {% if trips %}
  <div class="table-responsive">
    <table class="table table-hover align-middle">
      <thead class="table-light">
        <tr>
          <th>Photo</th>
          <th>Destination</th>
          <th>Created</th>
          <th>Budget</th>
          <th>Days</th>
          <th>Type</th>
          <th></th>
        </tr>
      </thead>
      <tbody>
        {% for trip in trips %}
        <tr>
          <td>
            {% if trip.image %}
              <img src="{{ trip.image.url }}" alt="Trip Photo" class="rounded" style="width: 60px; height: 40px; object-
fit: cover;">
            {% else %}
              <div class="bg-light rounded d-flex align-items-center justify-content-center" style="width: 60px; height:
40px;">
                <span class="text-muted small">📷</span>
              </div>
            {% endif %}
          </td>
          <td>{{ trip.destination }}</td>
          <td>{{ trip.created_at|date:"M d, Y H:i" }}</td>
          <td>₹{{ trip.budget|intcomma }}</td>
          <td>{{ trip.days }}</td>
          <td>{{ trip.travel_type|title }}</td>
          <td class="text-end">
            <a href="{% url 'trip-detail' trip.id %}" class="btn btn-sm btn-outline-primary me-1">View</a>
```

```
          <form method="post" action="{% url 'delete-trip' trip.id %}" class="d-inline" onsubmit="return confirm('Are you sure you want to delete this trip?')">
            {% csrf_token %}
            <button type="submit" class="btn btn-sm btn-outline-danger">Delete</button>
          </form>
        </td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
 </div>
 {% else %}
 <div class="card border-0 shadow-sm">
  <div class="card-body text-center">
   <h5 class="card-title">No saved trips yet</h5>
   <p class="card-text text-muted">Plan a trip to see it listed here.</p>
   <a href="{% url 'plan-trip' %}" class="btn btn-primary">Start Planning</a>
  </div>
 </div>
 {% endif %}

 {% if deleted_trips %}
 <div class="mt-5">
  <h4 class="h5 text-muted mb-3">🗑️ Recently Deleted Trips</h4>
  <div class="table-responsive">
   <table class="table table-hover align-middle opacity-75">
    <thead class="table-light">
     <tr>
      <th>Photo</th>
      <th>Destination</th>
      <th>Deleted</th>
      <th>Budget</th>
      <th>Days</th>
      <th>Type</th>
      <th></th>
     </tr>
    </thead>
    <tbody>
     {% for trip in deleted_trips %}
     <tr>
      <td>
       {% if trip.image %}
        <img src="{{ trip.image.url }}" alt="Trip Photo" class="rounded opacity-50" style="width: 60px; height: 40px; object-fit: cover;">
       {% else %}
        <div class="bg-light rounded d-flex align-items-center justify-content-center opacity-50" style="width: 60px; height: 40px;">
         <span class="text-muted small">📷</span>
        </div>
       {% endif %}
      </td>
```

```
          <td class="text-muted">{{ trip.destination }}</td>
          <td class="text-muted">{{ trip.deleted_at|date:"M d, Y H:i" }}</td>
          <td class="text-muted">₹{{ trip.budget|intcomma }}</td>
          <td class="text-muted">{{ trip.days }}</td>
          <td class="text-muted">{{ trip.travel_type|title }}</td>
          <td class="text-end">
            <form method="post" action="{% url 'recycle-trip' trip.id %}" class="d-inline">
              {% csrf_token %}
              <button type="submit" class="btn btn-sm btn-outline-success">Recycle</button>
            </form>
          </td>
        </tr>
        {% endfor %}
      </tbody>
    </table>
  </div>
 </div>
 {% endif %}
</div>
{% endblock %}
```

APPENDIX B: CALIBRATION AND SETUP GUIDE

1. Prerequisites

Before you start, ensure you have the following installed on your system:

- Python (version 3.8 or higher)
- PostgreSQL (running locally on port 5432)
- Git (optional, for version control)

2. Database Setup

This project uses PostgreSQL. You need to create a database and user matching the configuration in config/settings.py.

1. Open your terminal or command prompt.
2. Log into the PostgreSQL interactive terminal:

```
psql -U postgres
```

4. Create the database:

```
CREATE DATABASE ai_project_db;
```

6. Verify that the user postgres has the password postgres. If you need to alter the password:

```
ALTER USER postgres WITH PASSWORD 'postgres';
```

8. Exit the PostgreSQL terminal:

```
\q
```

3. Environment Setup

It's recommended to run the project inside an isolated virtual environment. The project is already configured with an environment directory named myenv.

1. Navigate to the project root directory:

```
2. cd /path/to/AI_Project
```

3. Activate the virtual environment:
   - On macOS/Linux:

```
o source myenv/bin/activate
```

   - On Windows:

```
o myenv\Scripts\activate
```

4. Install Dependencies

A requirements.txt file is present in the project directory with all the required dependencies.

With the virtual environment activated, install the required packages using pip:

```
pip install -r requirements.txt
```

5. Apply Migrations

Set up the database schema by applying the Django migrations:

```
python manage.py makemigrations
python manage.py migrate
```

6. Create a Superuser (Optional but Recommended)

To access the Django admin panel, you'll need an administrative user:

```
python manage.py createsuperuser
```

Follow the prompts to set your username, email, and password.

7. Run the Development Server

Finally, start the local server:

```
python manage.py runserver
```

The application should now be accessible at http://127.0.0.1:8000/.

You can log into the admin interface at http://127.0.0.1:8000/admin/ using the superuser credentials you created.

Note: The system is set up to look for static files in the static and staticfiles directories. If you make any CSS or JS changes, they will be loaded dynamically in development mode.